



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Zintegrowany System Zarządzania Harmonogramem Pracy dla Działów HR
i Pracowników "HRmonogram"

Bartosz BIEŃKOWSKI

Nr albumu: ⟨303786⟩

Kierunek: ⟨Informatyka⟩

Specjalność: ⟨Aplikacje mobilne, gry i multimedia⟩

PROWADZĄCY PRACĘ

⟨dr inż. Marcin Sobota⟩

WYDZIAŁ MATEMATYKI STOSOWANEJ

Gliwice 2026

Tytuł pracy

Zintegrowany System Zarządzania Harmonogramem Pracy dla Działów HR i Pracowników “HRmonogram”

Streszczenie

Aplikacja internetowa „HRmonogram” jest narzędziem wspomagającym zarządzanie zasobami ludzkimi oraz organizację czasu pracy w przedsiębiorstwach wielooddziałowych. System umożliwia efektywne tworzenie i edycję grafików pracy, zarządzanie strukturą organizacyjną firmy (obiekty, działy) oraz ewidencję pracowników i ich kontraktów. Aplikacja została zaprojektowana z myślą o zautomatyzowaniu procesów HR, ułatwieniu pracy menedżerom poprzez definiowanie preferencji zmianowych oraz zapewnieniu pracownikom przejrzystego wglądu w ich harmonogram. W pracy zostały przedstawione wymagania funkcjonalne i нефункционалне, projekt relacyjnej bazy danych oraz szczegóły implementacji systemu z wykorzystaniem nowoczesnych technologii webowych.

Słowa kluczowe

React, Flask, Python, zarządzanie personelem

Thesis title

Integrated Work Schedule Management System for HR Departments and Employees “HRmonogram”

Abstract

The web application “HRmonogram” is a tool supporting human resource management and work time organization in multi-branch enterprises. The system enables effective creation and editing of work schedules, management of the company’s organizational structure (objects, departments), and record-keeping of employees and their contracts. The application is designed to automate HR processes, facilitate managers’ work by defining shift preferences, and provide employees with transparent access to their schedules. The thesis presents functional and non-functional requirements, the design of the relational database, and details of the system implementation using modern web technologies.

Key words

React, Flask, Python, personnel management

Spis treści

Wstęp	1
Wprowadzenie do tematyki pracy	1
Cel i zakres pracy	2
Struktura pracy	3
1 Analiza tematu	5
1.1 Przegląd rozwiązań rynkowych	5
1.1.1 Grafik Optymalny	5
1.1.2 When I Work	7
1.2 Grupy docelowe i charakterystyka użytkowników	8
1.2.1 Podsumowanie ról w systemie	9
2 Projekt	11
2.1 Wymagania funkcjonalne	11
2.2 Wymagania niefunkcjonalne	12
2.3 Architektura aplikacji	13
2.3.1 Warstwa Bazy Danych (Database Layer)	13
2.3.2 Warstwa dostępu do danych (data access layer)	14
2.3.3 Warstwa logiki biznesowej(Business logic layer)	14
2.3.4 Warstwa prezentacji (Frontend Layer)	16
2.4 Diagram przypadków użycia aplikacji (UML)	18
2.4.1 Identyfikacja aktorów	18
2.4.2 Scenariusze przypadków użycia	19
3 Narzędzia i technologie	25
3.1 Technologie warstwy klienckiej (Frontend)	25
3.1.1 React.js	25
3.1.2 JavaScript (ES6+)	25
3.1.3 React Router DOM	26
3.1.4 HTML5 i CSS3	26
3.1.5 Fetch API	26

3.1.6	Jwt-decode	26
3.2	Technologie warstwy serwerowej	27
3.2.1	Flask	27
3.2.2	Rozszerzenia frameworka Flask	27
3.2.3	Node.js i NPM	28
3.3	Warstwa bazy danych	28
3.3.1	Architektura bezserwerowa (Serverless)	28
3.3.2	Przenośność (Portability)	28
3.3.3	Pełna obsługa standardu SQL i ACID	29
3.3.4	Wydajność w skali projektu	29
3.4	Narzędzia wspomagające proces wytwórczy	29
3.4.1	Środowisko programistyczne (IDE) – Visual Studio Code	29
3.4.2	System kontroli wersji – Git i GitHub	29
4	Implementacja	31
4.1	Technologie i narzędzia	31
4.1.1	Baza danych	31
4.1.2	Backend	32
4.1.3	Frontend	33
4.2	Struktura projektu	33
4.2.1	Struktura projektu backendu (Flask)	34
4.2.2	Struktura projektu frontend (React)	36
4.3	Implementacja warstwy danych	38
4.3.1	Modele danych i kluczowe encje	38
4.3.2	Relacje między tabelami	40
4.4	Obsługa routingu i interfejs API	40
4.4.1	Organizacja routingu (Blueprints)	40
4.4.2	Zestawienie punktów końcowych (Endpoints)	41
4.4.3	Format wymiany danych i kody statusu	41
4.5	Mechanizmy bezpieczeństwa i autoryzacji	43
4.6	Kluczowe algorytmy	43
4.6.1	Walidator Kodeksu Pracy	43
4.6.2	Algorytm wykrywania konfliktów czasowych	44
5	Testowanie	45
5.1	Środowisko i strategia testów	45
5.2	Testy jednostkowe (Unit Testing)	45
5.3	Testy integracyjne (Integration Testing)	46
5.4	Testy eksploracyjne	49

6	Interfejs użytkownika	51
7	Podsumowanie i wnioski	53
	Bibliografia	55
	Źródła	57
	Lista dodatkowych plików, uzupełniających tekst pracy	59
	Spis rysunków	61
	Spis tabel	63

Wstęp

Wprowadzenie do tematyki pracy

Współcześnie zarządzanie przedsiębiorstwem, niezależnie od jego skali, wymaga precyzyjnego planowania. Jednym z najbardziej kluczowych obszarów, a jednocześnie najtrudniejszym w strukturze organizacyjnej firm, jest właśnie zarządzanie zasobami ludzkimi (ang. *Human Resources* – HR). W dobie postępującej cyfryzacji oraz dynamicznych zmian na rynku, prowadzenie tradycyjnego harmonogramu czasu pracy, opartego na papierowych tabelach czy listach obecności, staje się mało efektywne. Ewidencja prowadzona w ten sposób jest dużo bardziej podatna na błąd ludzki. Cechuje się też słabą skalowalnością, ze względu na jej czasochłonność, czy też utrudnionym przepływem informacji pomiędzy kadrą zarządzającą a pracownikami.

Kolejnym nieoptymalnym typem narzędzi planistycznych używanym przez wiele organizacji, nawet takich zatrudniających dziesiątki osób, są arkusze kalkulacyjne, takie jak Microsoft Excel. Choć są one powszechnie dostępne, znane wielu użytkownikom i łatwe w obsłudze, w przypadku tworzenia złożonych harmonogramów ujawniają one swoje istotne ograniczenia. Praca z „Excelem” często sprowadza się do manualnego kopiowania danych, co obarczone jest wysokim ryzykiem błędu ludzkiego. Arkusze te zazwyczaj nie posiadają wbudowanych mechanizmów walidacji, które automatycznie ostrzegalyby przed naruszeniem norm czasu pracy czy dublowaniem zmian. Ponadto, model ten utrudnia pracę współdzieloną – przesyłanie plików drogą mailową prowadzi do problemów z wersjonowaniem (tzw. „Excel hell”) i brakiem jednego, spójnego źródła prawdy o grafiku.

W dziedzinie inżynierii oprogramowania, a w szczególności w projektowaniu systemów informatycznych o architekturze rozproszonej, stanowi to znane wyzwanie. Jest to związane z przejściem procesów biznesowych z modelu opartego na statycznych plikach lokalnych, takich jak arkusze kalkulacyjne, do modelu dynamicznych aplikacji internetowych, które działają na centralnej bazie danych. W środowisku, w którym duża liczba użytkowników odczytuje i modyfikuje te same zasoby, zapewnienie integralności i spójności danych jest kluczowym wyzwaniem. Pakiety biurowe, takie jak Excel, nie oferują natywnych mechanizmów kontroli współbieżności, transakcyjności lub zaawansowanego zarządzania uprawnieniami (RBAC — Zarządzanie uprawnieniami opartymi na rolach), co stwarza zagrożenie utraty danych lub nieautoryzowanego dostępu w całym przedsię-

biorstwie.

Z punktu widzenia systemów wspomagania decyzji (DSS), tworzenie harmonogramu pracy jest trudnym procesem optymalizacji, który wymaga przetworzenia wielu zmiennych o różnych priorytetach. Należy rozróżnić ograniczenia twarde (wymogi prawne, standardy Kodeksu Pracy, limity godzinowe w kontraktach) i miękkie (preferencje pracowników, porady menedżerów), których naruszenie jest dopuszczalne. Standardowe narzędzia biurowe nie mają logiki biznesowej, która może automatycznie potwierdzać te zależności w czasie rzeczywistym. Przeniesienie tego procesu do odpowiedniej aplikacji umożliwi wdrożenie algorytmów walidacyjnych, które na bieżąco wspierają decydenta lub menedżera, blokując możliwość wprowadzenia zmian, które nie są zgodne z założonymi regułami.

Ponadto współczesne wymagania operacyjne wymagają dostępu do danych w trybie rzeczywistym. Dystrybucja grafiku odbywa się poprzez wysyłkę plików w modelu tradycyjnym, co powoduje opóźnienia i problemy z wersjonowaniem dokumentów. Te przeszkody są usuwane przez specjalnie zaprojektowane narzędzie webowe. Zmiana wprowadzona przez administratora jest natychmiast widoczna na urządzeniu pracownika. Jest to funkcja wykraczająca poza standardowe możliwości pakietów biurowych i ma kluczowe znaczenie dla płynności operacyjnej współczesnych firm.

Cel i zakres pracy

W odpowiedzi na powyższe wyzwania, celem niniejszej pracy jest opracowanie i wdrożenie autorskiej aplikacji internetowej o nazwie „HRmonogram”. System ten ma służyć jako kompleksowe narzędzie, które pomoże menedżerom i działom HR skutecznie zarządzać kapitałem ludzkim i optymalizuje czas pracy w firmach o strukturze wielooddziałowej. Nadrzędnym założeniem projektowym jest stworzenie środowiska, które eliminuje wady rozwiązań tradycyjnych (papierowych i opartych na arkuszach kalkulacyjnych) poprzez centralizację danych oraz automatyzację powtarzalnych procesów planistycznych.

Realizacja celu głównego wymagała osiągnięcia szeregu celów szczegółowych, do których należą:

- **Cyfryzacja struktury organizacyjnej:** Tworzenie modelu danych pozwalającego na dokładne odwzorowanie hierarchii firmy, w tym jej podziału na fizyczne obiekty, takie jak oddziały lub placówki, oraz działy operacyjne znajdujące się w nich.
- **Implementacja logiki walidacyjnej:** Tworzenie mechanizmów kontrolnych, zarówno frontendowych, jak i backendowych, które w czasie rzeczywistym sprawdzają, czy harmonogramy są poprawne. Zapobiega to konfliktom, takim jak dublowanie zmian lub naruszenia zdefiniowanych reguł dostępności pracowników.
- **Zapewnienie bezpieczeństwa i granulacji dostępu:** Wdrożenie systemu autoryzacji opartego na rolach (RBAC) gwarantuje, że wrażliwe dane osobowe i kon-

traktowe są dostępne tylko dla uprawnionych użytkowników (Administrator, HR), podczas gdy większość pracowników ma dostęp tylko do swoich własnych harmonogramów.

- **Poprawa ergonomii pracy:** Tworzenie przyjaznego dla użytkownika graficznego interfejsu (GUI), który umożliwia szybkie tworzenie grafików pracy metodą wizualną i zapewnia łatwy dostęp do informacji z poziomu przeglądarki internetowej.

Zakres zadań obejmuje analizę wymagań funkcjonalnych i нефункциональных, poprzez projekt architektury systemu i modelu bazy danych, aż po implementację kodu źródłowego i weryfikację poprawności działania aplikacji.

Od strony technologicznej projekt został zrealizowany w architekturze klient-serwer, a jego zakres został podzielony na trzy kluczowe warstwy:

- **Warstwa danych:** Zaprojektowanie relacyjnej bazy danych (z wykorzystaniem silnika SQLite).
- **Warstwa serwerowa (Backend):** Implementacja logiki biznesowej przy użyciu języka Python oraz frameworka Flask. W ramach tej warstwy stworzono interfejs programistyczny aplikacji (API REST), odpowiedzialny za komunikację z bazą danych, obsługę żądań HTTP, uwierzytelnianie użytkowników z wykorzystaniem tokenów JWT (JSON Web Token) oraz przetwarzanie algorytmów walidujących grafiki pracy.
- **Warstwa prezentacji (Frontend):** Budowa responsywnej aplikacji klienckiej (SPA – Single Page Application) w oparciu o bibliotekę React. Zakres prac w tym obszarze obejmował stworzenie komponentów interfejsu, implementację mechanizmów zarządzania stanem aplikacji oraz integrację z API serwerowym w celu dynamicznego pobierania i wysyłania danych.
- **Zakres funkcjonalny pracy** obejmuje realizację modułów umożliwiających zarządzanie kontami użytkowników i ich kontraktami, konfigurację obiektów i preferencji zmianowych, tworzenie i modyfikacje harmonogramów pracy oraz prezentowanie indywidualnych planów pracy dla zalogowanych pracowników. Całość rozwiązania została zaprojektowana z uwzględnieniem zasad User Experience (UX), aby zminimalizować próg wejścia dla nowych użytkowników systemu.

Struktura pracy

Praca została podzielona na sześć rozdziałów, które logicznie odzwierciedlają proces powstawania oprogramowania – od analizy teoretycznej, przez projektowanie i implementację, aż po weryfikację i prezentację efektów końcowych.

Rozdział 1: Wstęp

Wprowadzenie do tematyki zarządzania czasem pracy, zdefiniowanie problemu badawczego, opis celu głównego i celów szczegółowych oraz zakresu projektu.

Rozdział 2: Analiza i projekt systemu

Przedstawia przegląd rozwiązań dostępnych na rynku, zdefiniowane wymagania funkcjonalne i нефункционалне, architekturę systemu w modelu klient-serwer oraz projekt relacyjnej bazy danych.

Rozdział 3: Implementacja

Opisuje technologie i narzędzia wykorzystane w projekcie (React, Flask, SQLite), strukturę kodu źródłowego oraz realizację kluczowych modułów, takich jak mechanizm autoryzacji czy algorytmy walidacji grafików.

Rozdział 4: Testowanie

Przedstawia proces weryfikacji poprawności działania aplikacji, w tym scenariusze testowe, testy manualne funkcjonalności oraz obsługę błędów.

Rozdział 5: Interfejs użytkownika

Prezentuje finalny wygląd aplikacji, omawia ergonomię rozwiązania oraz kluczowe widoki, takie jak panel logowania, interaktywny kalendarz pracy, zarządzanie strukturą firmy oraz edytor harmonogramów.

Rozdział 6: Podsumowanie

Podsumowuje etap realizacji pracy, ocenia stopień osiągnięcia założonych celów oraz przedstawia możliwości dalszego rozwoju aplikacji w przyszłości.

Rozdział 1

Analiza tematu

Kluczowym etapem przed procesem projektowania i wdrażania systemu informatycznego jest dokładna analiza problemu i środowiska biznesowego. W przypadku aplikacji HRmonogram, której celem jest usprawnienie procesów zarządzania zasobami ludzkimi, konieczne jest nie tylko zrozumienie specyfiki planowania czasu pracy; niezbędne jest również zbadanie istniejących alternatyw oraz dokładne określenie odbiorców systemu.

Niniejszy rozdział skupia się na dwóch istotnych aspektach przygotowawczych. Pierwszym krokiem było określenie niszy rynkowej dla realizowanego projektu poprzez rozważenie wad i zalet dostępnych na rynku rozwiązań. W drugiej części dokonano identyfikacji i charakterystyki grup docelowych, definiując ich role oraz zakres odpowiedzialności. To dało podstawę do opracowania wymagań funkcjonalnych w następnym etapie pracy.

1.1 Przegląd rozwiązań rynkowych

Rynek oprogramowania klasy Workforce Management (WFM) charakteryzuje się obecnie dużą dywersyfikacją dostępnych narzędzi. Aby właściwie umiejscowić projekt „HRmonogram” w obecnych realiach rynkowych, przeanalizowano dwa systemy reprezentujące odmienne filozofie projektowe: rodzimą platformę „Grafik Optymalny” oraz globalne rozwiązanie „When I Work”. Poniższe zestawienie koncentruje się na funkcjonalności, modelach biznesowych oraz mocnych i słabych stronach w kontekście potrzeb małych i średnich przedsiębiorstw.

1.1.1 Grafik Optymalny

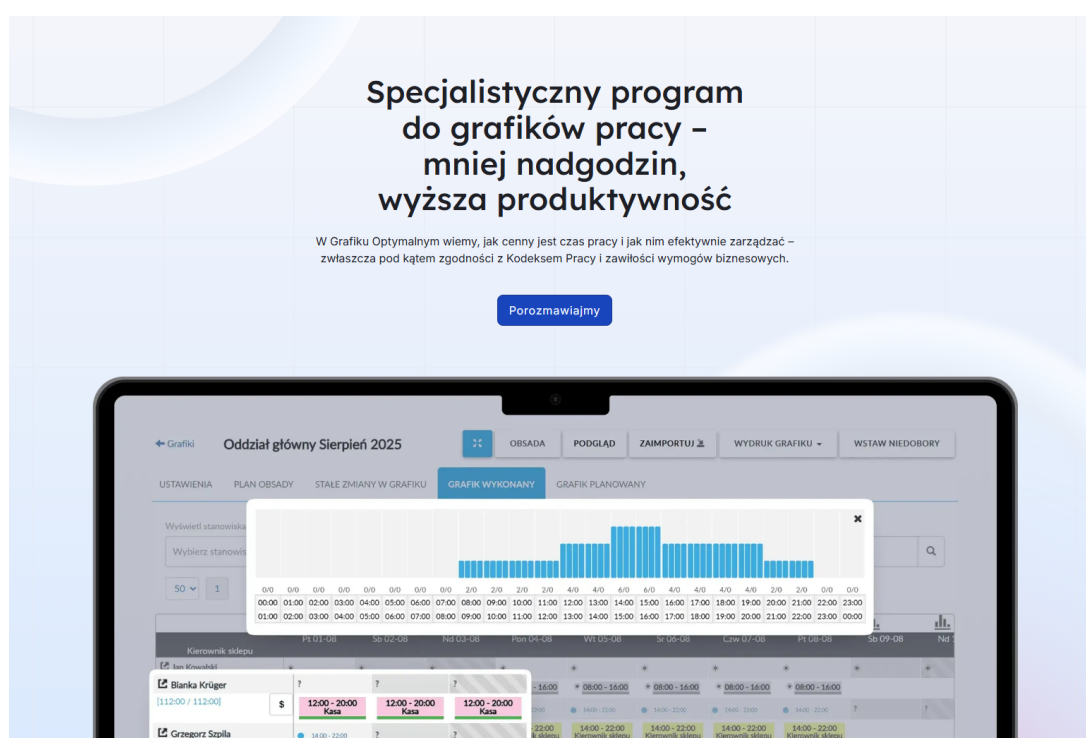
Specyfika rozwiązania

Grafik Optymalny to narzędzie zaprojektowane stricte pod kątem polskiego porządku prawnego. Architektura systemu została podporządkowana wymogom Kodeksu Pracy, co czyni go rozwiązaniem pierwszego wyboru dla podmiotów, gdzie priorytetem jest bezpieczeństwo prawne. Platforma, działająca w modelu SaaS, wykorzystuje algorytmy automa-

tyzujące proces harmonogramowania, uwzględniając przy tym skomplikowane zmienne, takie jak doby pracownicze, normy odpoczynku czy wymiary etatów. Pozostałe funkcjonalności obejmują automatyczne generowanie grafików, elektroniczny obieg wniosków urlopowych, ewidencję czasu pracy (RCP) oraz rozbudowane raportowanie dla kadr i płac.

Model kosztowy

Polityka cenowa opiera się na miesięcznej subskrypcji uzależnionej od liczby pracowników. Warto odnotować brak bezpłatnej wersji do użytku komercyjnego (dostępny jest jedynie okres testowy). Koszt implementacji w podstawowych wariantach oscyluje w granicach 10–15 PLN netto za pracownika, rosnąc proporcjonalnie do stopnia zaawansowania funkcji (np. integracje API).



Rysunek 1.1: Grafik Optymalny (Źródło: <https://www.grafikoptymalny.pl/> , Data dostępu: 5.12.2025)

Zalety:

- **Kompatybilność prawna:** Wbudowane reguły walidacyjne oparte na Kodeksie Pracy stanowią „bezpiecznik” chroniący pracodawcę przed planowaniem naruszającym przepisy.
- **Efektywność algorytmiczna:** Możliwość automatycznego generowania harmonogramów przy uwzględnieniu dostępności zasobów ludzkich znacząco redukuje czasochłonność procesów zarządczych.

- **Ekosystem integracji:** System zapewnia płynną wymianę danych z wiodącym oprogramowaniem kadrowo-płacowym w Polsce (m.in. Enova, Optima).

Wady:

- **Krzywa uczenia się:** Rozbudowana parametryzacja prawna skutkuje wysokim progiem wejścia; interfejs może okazać się mało intuicyjny dla mniej zaawansowanych użytkowników.
- **Bariera ekonomiczna dla mikroprzedsiębiorstw:** Brak planu *freemium* czyni to rozwiązanie stosunkowo kosztownym dla najmniejszych podmiotów.

1.1.2 When I Work

Specyfika rozwiązania

Amerykańska platforma When I Work reprezentuje podejście zorientowane na *User Experience* (UX) oraz płynność komunikacji wewnątrzzespołowej. W odróżnieniu od polskiego odpowiednika, system ten stawia na prostotę i nowoczesny design. Interfejs oparty na mechanizmie *drag & drop* oraz silna koncentracja na rozwiązaniach mobilnych mają na celu maksymalne uproszczenie interakcji na linii menedżer–pracownik. Aplikacja umożliwia nie tylko wgląd w grafik, ale także autonomiczną wymianę zmian między pracownikami oraz komunikację poprzez wbudowany czat.

Model kosztowy

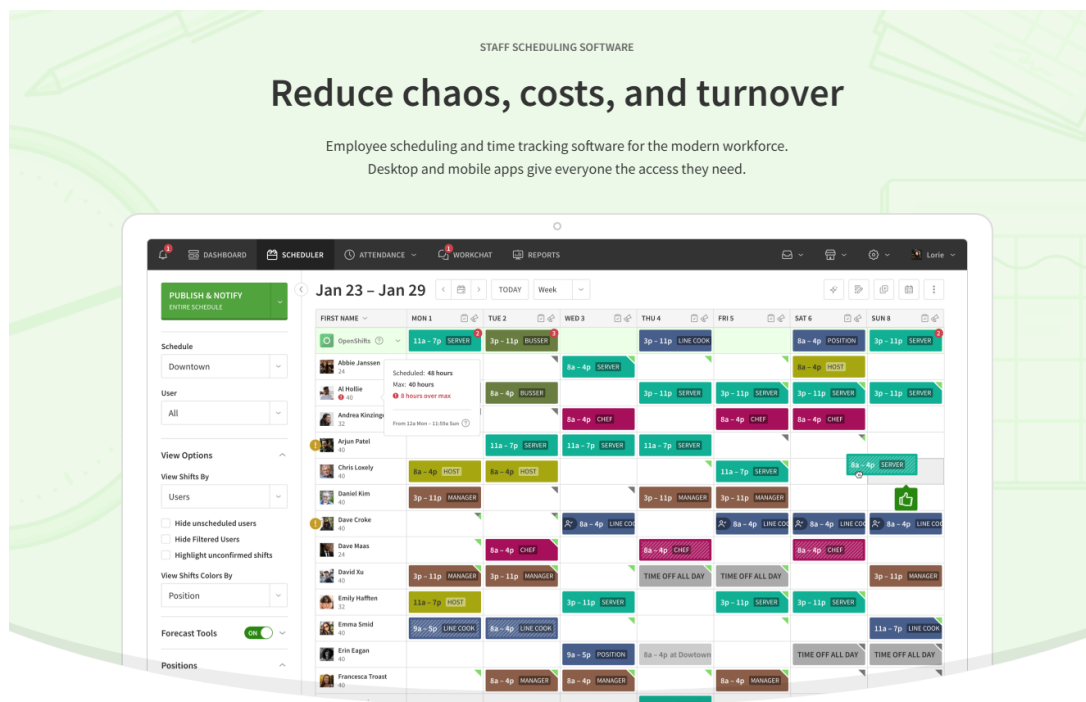
Rozwiązanie wyróżnia się modelem *Freemium*, umożliwiającym małym zespołom bezpłatne korzystanie z podstawowych funkcjonalności. Wersje płatne (Standard, Advanced) rozliczane są subskrypcyjnie, ceny zaczynają się od około 2,50–4,00 USD za użytkownika miesięcznie.

Zalety:

- **Użyteczność (Usability):** Intuicyjny interfejs eliminuje konieczność przeprowadzania szkoleń wdrożeniowych.
- **Mobilność:** Wysoko oceniana aplikacja mobilna wspiera zarządzanie operacyjne w czasie rzeczywistym.
- **Elastyczność operacyjna:** Funkcja *shift swapping* (wymiana zmian) zwiększa autonomię pracowników i odciąża kadrę menedżerską.

Wady:

- **Niedostosowanie do krajowej legislacji:** Brak walidatorów zgodnych z polskim Kodeksem Pracy (np. w zakresie norm średniotygodniowych) wymusza ręczną weryfikację grafików pod kątem prawnym.



Rysunek 1.2: When I Work (Źródło: <https://wheniwork.com/>, Data dostępu: 5.12.2025)

- **Bariera językowa:** Anglojęzyczny interfejs oraz wsparcie techniczne mogą stanowić istotną przeszkodę w organizacjach zatrudniających personel nieposługujący się biegle tym językiem.

1.2 Grupy docelowe i charakterystyka użytkowników

System „HRmonogram” jest rozwiązaniem wielodostępowym, zaprojektowanym do obsługi procesów na różnych szczeblach struktury organizacyjnej przedsiębiorstwa. Wyróżnienie grup docelowych oraz zdefiniowanie ról użytkowników jest kluczowe dla zapewnienia bezpieczeństwa danych oraz ergonomii pracy. W projektowanym systemie zdefiniowano cztery główne grupy użytkowników, różniące się zakresem uprawnień oraz dostępnymi funkcjami aplikacji.

1. Administrator Systemu (IT / Admin)

Użytkownik o najwyższym poziomie uprawnień, odpowiedzialny za techniczną sprawność działania systemu. Rola ta nie jest związana z operacyjnym zarządzaniem kadrami, lecz z utrzymaniem infrastruktury aplikacji.

- **Charakterystyka:** Osoba posiadająca wiedzę techniczną.
- **Główne zadania:** Zarządzanie kontami użytkowników globalnych (Global HR), resetowanie haseł w sytuacjach awaryjnych, nadzór nad poprawnością działania bazy danych oraz konfiguracja parametrów systemowych.

2. Global HR (Zarządzanie centralne)

Reprezentuje kadrę zarządzającą przedsiębiorstwa (np. Dyrektor Operacyjny, Główny HR Manager). Użytkownik ten jest architektem struktury organizacyjnej w systemie.

- **Charakterystyka:** Użytkownik podejmujący decyzje strategiczne, definiujący standardy obowiązujące we wszystkich oddziałach firmy.
- **Główne zadania:** Tworzenie i edycja obiektów (filii, oddziałów) oraz działów wewnętrznych, definiowanie słowników systemowych (np. typy kontraktów, wymiary etatów), nadawanie uprawnień menedżerom lokalnym (Local HR) oraz wgląd w zbiorcze dane z całej organizacji.

3. Local HR (Menedżer Obiektu / Kierownik)

Kluczowa grupa docelowa aplikacji – osoby, które na co dzień borykają się z problemem układania grafików. Rola ta przypisana jest do konkretnego obiektu (np. kierownik restauracji, kierownik sklepu).

- **Charakterystyka:** Użytkownik operacyjny. Wymaga narzędzia o przejrzystym interfejsie, które szybko zweryfikuje poprawność harmonogramu i pozwoli na elastyczne zmiany w planie.
- **Główne zadania:** Tworzenie harmonogramów pracy dla swojego zespołu, edycja szablonów zmian (godziny otwarcia/zamknięcia), weryfikacja dostępności pracowników, zatwierdzanie grafików oraz zarządzanie profilem pracowników w obrębie swojego obiektu.

4. Pracownik (Użytkownik końcowy)

Najliczniejsza grupa użytkowników systemu. Są to osoby zatrudnione w przedsiębiorstwie, dla których aplikacja jest źródłem informacji o czasie pracy.

- **Charakterystyka:** Użytkownik korzystający z systemu głównie biernie, często za pomocą urządzeń mobilnych (smartfonów). Oczekuje prostego i czytelnego interfejsu.
- **Główne zadania:** Logowanie do systemu w celu sprawdzenia własnego harmonogramu pracy na nadchodzący okres.

1.2.1 Podsumowanie ról w systemie

Poniższy schemat obrazuje hierarchię dostępu do danych w systemie „HRmonogram”:

Administrator: Dostęp techniczny do całości systemu.

Global HR: Dostęp do edycji struktury firmy i wszystkich pracowników.

Local HR: Dostęp do edycji grafików i pracowników tylko we własnym obiekcie.

Pracownik: Dostęp wyłącznie do odczytu własnych danych i własnego grafiku (brak wglądu w dane innych osób ze względu na RODO).

Rozdział 2

Projekt

Niniejszy rozdział poświęcony jest technicznym specyfikacjom aplikacji HRmonogram. Jest to przejście od wcześniej zidentyfikowanych potrzeb biznesowych do konkretnego planu ich wdrożenia. Podczas gdy poprzednia część pracy określała problem, ta część skupia się na tym, jak go rozwiązać.

Najpierw określono wymagania funkcjonalne i нефункционалне, które stanowią podstawę działania systemu oraz oczekiwane standardy jakościowe. Po tym przedstawiono architekturę rozwiązania, która obejmuje podział na warstwy logiczne i strukturę klient-serwer. Uzasadniono również dobór wzorców projektowych, które mają na celu zapewnienie przejrzystości i skalowalności kodu. Na końcu przedstawiono także przypadki użycia aplikacji.

2.1 Wymagania funkcjonalne

Użytkownik (Wspólne dla wszystkich ról):

- Ma możliwość bezpiecznego logowania się do aplikacji przy użyciu adresu e-mail i hasła.
- Ma możliwość wylogowania się z systemu, kończąc sesję pracy.

Pracownik:

- Ma możliwość przeglądania własnego harmonogramu pracy w widoku kalendarza.
- Ma możliwość sprawdzenia szczegółów zmian
- Ma możliwość podglądu podsumowania swoich godzin pracy w danym miesiącu.

Local HR (Menedżer Obiektu):

- Ma możliwość tworzenia nowych harmonogramów pracy dla przypisanego obiektu.

- Ma możliwość dodawania, edytowania i usuwania zmian w grafiku metodą wizualną.
- Ma możliwość korzystania z gotowych szablonów zmian (np. zmiany dzienne/nocne) w celu przyspieszenia planowania.
- Ma możliwość przeglądania listy pracowników dostępnych w danym obiekcie.
- Ma możliwość weryfikacji poprawności grafiku (system posiada podstawową walidację).

Global HR / Administrator:

- Ma możliwość zarządzania całą strukturą organizacyjną firmy (dodawanie i edycja obiektów oraz działów).
- Ma możliwość zarządzania bazą pracowników (tworzenie kont, edycja danych osobowych i ról).
- Ma możliwość definiowania typów kontraktów oraz wymiarów etatu.
- Ma możliwość podglądu i edycji harmonogramów we wszystkich obiektach w firmie.

2.2 Wymagania нефunkcjonalne

Bezpieczeństwo danych:

- Hasła użytkowników muszą być przechowywane w bazie danych wyłącznie w formie niejawnej.
- Dostęp do zasobów API (backendu) musi być zabezpieczony mechanizmem tokenów (np. JWT), aby uniemożliwić dostęp osobom niepowołanym.
- System musi blokować dostęp do paneli administracyjnych dla użytkowników o niższych uprawnieniach (Pracownik).

Kompatybilność i Dostępność:

- Aplikacja musi być dostępna z poziomu przeglądarki internetowej (np. Google Chrome, Mozilla Firefox, Safari) bez konieczności instalacji dodatkowego oprogramowania.
- Interfejs użytkownika musi być responsywny (RWD – Responsive Web Design), czyli automatycznie dostosowywać się do rozmiaru ekranu urządzenia (komputer, tablet, smartfon).

Wydajność i Niezawodność:

- System powinien zapewniać płynne działanie interfejsu (brak wyraźnych opóźnień) przy standardowym obciążeniu.
- Baza danych musi zapewniać integralność danych (np. spójność kluczy obcych), aby zapobiec powstawaniu błędnych rekordów w grafikach.

Ergonomia (User Experience):

- Interfejs aplikacji powinien być intuicyjny i minimalistyczny, aby czas wdrożenia nowego użytkownika był jak najkrótszy.
- Aplikacja powinna komunikować się z użytkownikiem w języku polskim.
- System powinien informować użytkownika o błędach (np. niepoprawne hasło, brak dostępu) w sposób jasny i zrozumiały.

2.3 Architektura aplikacji

Aplikacja „HRmonogram” została zaprojektowana w oparciu o architekturę wielowarstwową (ang. *Multi-tier architecture*) realizowaną w modelu klient-serwer, co umożliwia separację odpowiedzialności (SoC – *Separation of Concerns*) pomiędzy warstwą prezentacji danych, logiką biznesową a warstwą przechowywania danych oraz sprawniejsze utrzymanie systemu.

2.3.1 Warstwa Bazy Danych (Database Layer)

Warstwa bazy danych odpowiedzialna jest za składowanie informacji oraz zapewnienie ich trwałości. Zaprojektowano ją w modelu relacyjnym (RDBMS – *Relational Database Management System*), co gwarantuje uporządkowaną strukturę danych oraz możliwość definiowania ścisłych relacji pomiędzy poszczególnymi obiektami systemu.

W projekcie jako silnik bazy danych wykorzystano SQLite. Jest to lekki system zarządzania bazą danych, zgodny ze standardem ACID (*Atomicity, Consistency, Isolation, Durability*). Wybór SQLite podyktowany był architekturą projektu – jako baza bezserwerowa (*serverless*), przechowuje ona całą strukturę oraz dane w pojedynczym pliku systemowym, co znacząco ułatwia przenoszenie aplikacji oraz tworzenie kopii zapasowych.

Model danych (Schemat)

Struktura bazy danych została zaprojektowana w oparciu o analizę wymagań funkcjonalnych. Do najważniejszych encji w systemie należą:

User (Użytkownik): Tabela przechowująca dane logowania (zhashowane hasło, email) oraz dane osobowe.

Role (Rola): Słownik ról systemowych (Admin, HR, Pracownik) determinujący uprawnienia.

Object & Department: Tabele odwzorowujące fizyczną strukturę firmy (Obiekty i Działy).

Contract: Tabela definiująca warunki zatrudnienia pracownika (wymiar etatu, typ umowy).

Shift (Zmiana): Centralna tabela systemu łącząca pracownika z konkretnym dniem, godzinami pracy oraz obiektem.

ShiftTemplate: Tabela szablonów zmian, usprawniająca proces planowania.

Pomiędzy powyższymi encjami zdefiniowano relacje typu jeden-do-wielu (np. jeden Obiekt posiada wiele Działów) oraz wiele-do-wielu, co zapewnia logiczną spójność przechowywanych informacji.

2.3.2 Warstwa dostępu do danych (data access layer)

Warstwa dostępu do danych jest pośrednikiem pomiędzy bazą danych a logiką biznesową aplikacji, dzięki czemu komunikacja przebiega w sposób bezpieczny i uporządkowany. W projekcie warstwę tę zrealizowano przy użyciu techniki mapowania obiektowo-relacyjnego (ORM – *Object-Relational Mapping*), co zapewnia większą klarowność danych i wydajność kodu. Wykorzystano również bibliotekę SQLAlchemy, która pozwala na mapowanie modeli danych na odpowiadające im tabele.

Bezpieczeństwo i transakcyjność

Zastosowanie dedykowanej warstwy dostępu do danych pełni również kluczową funkcję w zakresie bezpieczeństwa systemu:

- **Ochrona przed SQL Injection:** Biblioteka SQLAlchemy automatycznie określa zapytania wysyłane do bazy danych, co skutecznie neutralizuje próby implementacji złośliwego kodu SQL przez pola formularzy.
- **Zarządzanie sesją i transakcjami:** Warstwa DAL obsługuje mechanizm sesji, który grupuje operacje na danych w transakcje. Zmiany są zatwierdzane w bazie (*commit*) dopiero po poprawnym wykonaniu całego bloku operacji logicznych.

2.3.3 Warstwa logiki biznesowej (Business logic layer)

Warstwa logiki biznesowej stanowi „mózg” aplikacji „HRmonogram”. Zaimplementowano ją w języku Python przy użyciu frameworka Flask. Umożliwiło to precyzyjne odwzorowanie procesów HR w kodzie, zapewniając jednocześnie wysoką wydajność obsługi żądań HTTP.

W ramach tej warstwy zrealizowano kluczowe algorytmy sterujące działaniem systemu:

1. Zarządzanie tożsamością i bezpieczeństwem (*Authentication & Security*)

Ze względu na wrażliwość danych osobowych i kadrowych, implementacja logiki bezpieczeństwa była priorytetem.

- **Haszowanie haseł:** System nie przechowuje haseł w formie jawnej. Podczas rejestracji lub zmiany hasła, ciąg znaków jest przetwarzany przez algorytm kryptograficzny (z biblioteki `werkzeug.security`), a do bazy trafia jedynie bezpieczny skrót (*hash*). Podczas logowania hasło podane przez użytkownika jest haszowane i porównywane z tym w bazie.
- **Tokenizacja sesji (JWT):** Po pomyślnym zalogowaniu backend generuje i podpisuje cyfrowo token JWT (*JSON Web Token*). Token ten zawiera zakodowane identyfikatory użytkownika (`user_id`) oraz jego rolę (`role`). Dzięki temu serwer jest bezstanowy – nie musi pamiętać sesji w pamięci RAM, a jedynie weryfikuje poprawność podpisu tokena przy każdym zapytaniu.

2. Kontrola dostępu oparta na rolach (*RBAC Logic*)

Aplikacja wykorzystuje niestandardowe dekoratory (funkcje oplatające), takie jak `@role_required` czy `@admin_required`, które działają jako oprogramowanie pośredniczące (*middleware*). Przed wykonaniem jakiejkolwiek funkcji biznesowej (np. usunięcia pracownika), dekorator sprawdza, czy w nagłówku zapytania znajduje się ważny token oraz rolę zaszyfrowaną w tokenie (np. `ROLE_USER`). W przypadku próby wywołania tokenu administracyjnego przez zwykłego pracownika, następuje przerwanie procesu i zwracany jest błąd 403 `Forbidden`.

3. Algorytmy walidacji harmonogramu

Najbardziej złożona logika biznesowa dotyczy modułu planowania (*Shift Service*). Zanim nowa zmiana zostanie zapisana w bazie danych, backend uruchamia sekwencję walidatorów:

- **Wykrywanie kolizji czasowych:** System pobiera z bazy danych wszystkie istniejące zmiany danego pracownika i matematycznie sprawdza, czy proponowany przedział czasowy (Start–Koniec) nie pokrywa się z inną aktywnością. Zapobiega to fizycznemu przypisaniu jednej osoby do dwóch miejsc jednocześnie.
- **Weryfikacja przynależności:** Logika sprawdza, czy menedżer (Local HR) próbujący dodać zmianę, zarządza obiektem, do którego przypisany jest dany pracownik.

4. API i Serializacja danych

Komunikacja z frontendem odbywa się poprzez ściśle zdefiniowane punkty końcowe

(*Endpoints*). Ponieważ obiekty bazy danych (modele SQLAlchemy) nie mogą być bezpośrednio wysłane przez sieć, warstwa logiczna odpowiada za ich serializację. Zaimplementowano metody pomocnicze (np. `to_json`), które przekształcają obiekty Pythonowe w format JSON, jednocześnie filtrując dane wrażliwe (np. automatycznie usuwając pole hasła z obiektu `User` przed wysłaniem go do przeglądarki).

2.3.4 Warstwa prezentacji (Frontend Layer)

Warstwę prezentacji, stanowiącą interfejs użytkownika (GUI) systemu „HRmonogram”, zrealizowano w technologii React. Jest to nowoczesna biblioteka języka JavaScript, służąca do budowania dynamicznych interfejsów w architekturze SPA (*Single Page Application*). Wybór tego rozwiązania pozwolił na stworzenie aplikacji, która działa płynnie i nie wymaga przeładowywania całej strony przy każdej interakcji użytkownika, co znacząco podnosi komfort pracy (*User Experience*).

Główne aspekty implementacyjne warstwy prezentacji obejmują:

1. Architektura komponentowa

Interfejs aplikacji został podzielony na mniejsze, niezależne fragmenty kodu zwane komponentami. Takie podejście ułatwia zarządzanie kodem i pozwala na wielokrotne wykorzystanie tych samych elementów w różnych miejscach systemu. W projekcie wyróżniono dwa typy komponentów:

- **Komponenty strukturalne (Layout):** Odpowiadają za szkielet aplikacji, np. `Navbar` (pasek nawigacji), `Sidebar` (menu boczne) czy `Footer`.
- **Komponenty funkcjonalne (Features):** Realizują konkretne zadania biznesowe, np. `ScheduleTable` (tabela z grafikiem), `EmployeeForm` (formularz dodawania pracownika) czy `ShiftCard` (wizualna reprezentacja pojedynczej zmiany).

Kluczowe komponenty aplikacji

Warstwa prezentacji składa się z kilku głównych komponentów w procesie interakcji z użytkownikiem:

App (Główny kontener aplikacji): Jest to komponent nadrzędny (korzeń drzewa komponentów), który inicjalizuje całą aplikację. Jego główną rolą jest konfiguracja routingu (mapowania adresów URL na widoki) oraz dostarczenie globalnych kontekstów (np. stanu zalogowania) do wszystkich komponentów podrzędnych.

AuthContext (Dostawca tożsamości): Komponent logiczny (bez reprezentacji wizualnej), odpowiedzialny za przechowywanie i udostępnianie informacji o aktualnie zalogowanym użytkowniku. Zarządza tokenem JWT, sprawdza rolę użytkownika (Admin/User) i decyduje, które części interfejsu są dostępne, a które ukryte.

Navbar (Pasek nawigacyjny): Komponent dynamiczny, który zmienia swój wygląd w zależności od uprawnień użytkownika. Dla Administratora wyświetla zakładki do zarządzania strukturą i pracownikami, natomiast dla zwykłego pracownika ogranicza widok jedynie do „Mój grafik” i „Wyloguj”.

ScheduleGrid (Siatka harmonogramu): Centralny komponent wizualny systemu. Odpowiada za wygenerowanie tabeli kalendarza dla wybranego miesiąca. Jego rolą jest pobranie danych o zmianach z API, odpowiednie ich posortowanie i wyświetlenie w komórkach przypisanych do konkretnych dni i pracowników.

ShiftCard (Karta zmiany): Mniejszy komponent używany wielokrotnie wewnątrz ScheduleGrid. Reprezentuje pojedynczą zmianę (np. „08:00 - 16:00”). Obsługuje interakcje, takie jak kliknięcie w celu edycji zmiany lub wyświetlenia szczegółów przypisanego obiektu.

EmployeeForm (Edytor pracownika): Komponent formularzowy służący do dodawania i edycji danych kadrowych. Jego kluczową rolą jest walidacja wprowadzanych danych (np. sprawdzanie formatu e-mail, wymaganych pól) przed wysłaniem ich do serwera, co odciąża backend z podstawowej weryfikacji błędów.

2. Zarządzanie stanem i cyklem życia (Hooks)

Aplikacja wykorzystuje tzw. Hooki do zarządzania stanem komponentów logicznych:

useState: Służy do przechowywania lokalnych danych, takich jak wpisywana treść w formularzach, aktualnie wyświetlany miesiąc w kalendarzu czy stan widoczności okna modalnego. Zmiana stanu powoduje automatyczną aktualizację odpowiedniego fragmentu interfejsu.

useEffect: Odpowiada za operacje poboczne, takie jak pobranie listy pracowników z serwera w momencie załadowania komponentu.

useContext: Wykorzystany do globalnego zarządzania stanem zalogowanego użytkownika. Dzięki temu informacja o tym, kto jest zalogowany (i jakie ma uprawnienia), jest dostępna w każdym miejscu aplikacji bez konieczności przekazywania jej ręcznie przez wszystkie poziomy komponentów.

3. Routing po stronie klienta

Za nawigację wewnątrz aplikacji odpowiada biblioteka React Router. Umożliwia ona

mapowanie adresów URL (np. `/dashboard`, `/schedule`, `/settings`) na konkretne widoki, bez wysyłania żądania do serwera o nową stronę. Dzięki temu przechodzenie między zakładkami odbywa się natychmiastowo. Zaimplementowano tu również mechanizm *Protected Routes* – jeśli niezalogowany użytkownik spróbuje wejść pod adres `/admin`, router automatycznie przekieruje go na stronę logowania.

4. Komunikacja z API i obsługa JWT

Frontend pełni rolę odbiorcy API wystawionego przez warstwę backendową. Do komunikacji asynchronicznej (AJAX) wykorzystano mechanizm `fetch` lub bibliotekę `axios`. Kluczowym elementem jest tutaj obsługa autoryzacji:

- Podczas logowania frontend odbiera token JWT i zapisuje go bezpiecznie w pamięci przeglądarki (*Local Storage* lub *Session Storage*).
- Przy każdym kolejnym zapytaniu do serwera (np. „pobierz grafik”), aplikacja automatycznie dołącza ten token do nagłówka zapytania (`Authorization: Bearer <token>`). Pozwala to backendowi zidentyfikować użytkownika wysyłającego żądanie.

2.4 Diagram przypadków użycia aplikacji (UML)

Diagram przypadków użycia przedstawia wizualizację interakcji zachodzących pomiędzy użytkownikami a aplikacją oraz funkcjonalności dostępne dla poszczególnych ról.

2.4.1 Identyfikacja aktorów

Na diagramie wyszczególniono czterech głównych aktorów, których role pokrywają się z grupami docelowymi zdefiniowanymi w fazie analizy:

- **Pracownik:** Aktor o podstawowym poziomie uprawnień. Jego interakcja z systemem ogranicza się do operacji odczytu (przeglądanie harmonogramu).
- **Local HR (Menedżer):** Aktor odpowiedzialny za procesy operacyjne w obrębie danego obiektu. Rozszerza on funkcjonalność pracownika o możliwość edycji grafików.
- **Global HR:** Aktor zarządzający strukturą firmy.
- **Administrator:** Aktor techniczny, posiadający pełną kontrolę nad użytkownikami i systemem.

2.4.2 Scenariusze przypadków użycia

Przypadek użycia: Utworzenie konta pracownika

- **Aktor:** Global HR (lub Administrator).
- **Cel:** Rejestracja nowego użytkownika w systemie i przypisanie go do struktury.
- **Warunki początkowe:** Użytkownik jest zalogowany i posiada uprawnienia globalne (Administrator/Global HR).
- **Warunki końcowe:** Nowe konto jest aktywne, a pracownik otrzymuje hasło startowe.
- **Scenariusz:**
 1. Użytkownik wybiera z menu głównego zakładkę (Dodaj Użytkownika).
 2. System wyświetla formularz rejestracyjny.
 3. Użytkownik wypełnia pola obowiązkowe: Imię i Nazwisko, E-mail.
 4. Użytkownik definiuje hasło ręcznie lub klika przycisk **Generuj**, aby system stworzył bezpieczne hasło losowe, które musi zostać zmienione przy pierwszym logowaniu.
 5. Użytkownik wybiera z list rozwijanych: Rolę, Typ umowy, Obiekt oraz Departament.
 6. Użytkownik klika przycisk **Utwórz Użytkownika**.
 7. System sprawdza unikalność adresu e-mail.
 - *Adres e-mail jest wolny:*
 - (a) System hashuje hasło i tworzy rekord w bazie danych.
 - (b) System wyświetla komunikat (alert) z potwierdzeniem sukcesu oraz wyświetla tymczasowe hasło do przekazania pracownikowi.
 - (c) System przekierowuje użytkownika do listy pracowników.
 - *Adres e-mail jest już zajęty:*
 - (a) System wyświetla komunikat błędu informujący o istniejącym użytkowniku.

Przypadek użycia: Przeglądanie własnego harmonogramu

- **Aktor:** Pracownik.
- **Cel:** Sprawdzenie zaplanowanych godzin pracy na najbliższy okres.
- **Warunki początkowe:** Pracownik jest zalogowany do systemu.

- **Warunki końcowe:** Pracownik zapoznał się z grafikiem.
- **Scenariusz:**
 1. Użytkownik loguje się do aplikacji.
 2. System rozpoznaje rolę „Pracownik” lub „Local HR” i automatycznie ładuje (Pulpit Pracownika).
 3. System pobiera aktualnie opublikowany grafik dla obiektu przypisanego do pracownika.
 4. System wyświetla harmonogram w formie tabeli, gdzie kolumny reprezentują dni miesiąca, a wiersze pracowników.
 5. System automatycznie wyróżnia kolumnę z dzisiejszą datą oraz wiersz zalogowanego użytkownika.
 6. Użytkownik odczytuje symbol zmiany (np. „R”, „06-14”) w komórce tabeli.
 7. Użytkownik weryfikuje znaczenie symbolu w legendzie skrótów znajdującej się pod tabelą.
 8. Użytkownik przełącza zakładki (opcjonalnie): „Nadchodzące” lub „Historia”, aby zobaczyć inne grafiki.

Przypadek użycia: Utworzenie nowego obiektu (filii)

- **Aktor:** Global HR.
- **Cel:** Zdefiniowanie nowej placówki w strukturze organizacyjnej firmy.
- **Warunki początkowe:** Użytkownik jest zalogowany i posiada uprawnienia globalne.
- **Warunki końcowe:** Nowy obiekt jest dostępny w systemie i można do niego przypisywać pracowników oraz grafiki.
- **Scenariusz:**
 1. Użytkownik wybiera z menu zakładkę (Obiekty).
 2. System wyświetla listę istniejących obiektów oraz formularz dodawania.
 3. Użytkownik wprowadza „Nazwę obiektu” (np. Restauracja Centrum) oraz „Lokalizację”.
 4. Użytkownik klika przycisk Add Object.
 5. System weryfikuje dane.
 - *Dane są poprawne:*

- (a) System tworzy nowy rekord w tabeli `Objects`.
- (b) Nowy obiekt pojawia się natychmiastowo na liście poniżej formularza.
- *Wystąpił błąd (np. brak nazwy):*
 - (a) System nie dodaje obiektu i wyświetla stosowny komunikat błędu.

Przypadek użycia: Edycja danych i umowy pracownika

- **Aktor:** Global HR / Admin.
- **Cel:** Aktualizacja danych osobowych, przypisania do obiektu lub typu umowy.
- **Warunki początkowe:** Użytkownik jest zalogowany i znajduje się w widoku listy pracowników.
- **Warunki końcowe:** Rekord pracownika w bazie danych zostaje zaktualizowany.
- **Scenariusz:**
 1. Użytkownik przechodzi do zakładki „Pracownicy” (`Employees`).
 2. Użytkownik wyszukuje pracownika na liście i klika w jego wiersz.
 3. System wyświetla panel szczegółów pracownika (`UserDetails`).
 4. Użytkownik modyfikuje wybrane pola, np. zmienia „Obiekt”, „Departament” lub „Typ Umowy”.
 5. Użytkownik klika przycisk `Update User`.
 6. System waliduje poprawność danych.
 - *Dane są poprawne:*
 - (a) System aktualizuje rekord w bazie danych.
 - (b) System wyświetla komunikat „User updated successfully”.
 - *Dane są błędne:*
 - (a) System wyświetla komunikat o błędzie i nie zapisuje zmian.

Przypadek użycia: Tworzenie nowego harmonogramu pracy

- **Aktor:** Local HR (lub Global HR).
- **Cel:** Zaplanowanie zmian roboczych dla pracowników w danym miesiącu.
- **Warunki początkowe:** Użytkownik jest zalogowany i posiada uprawnienia do zarządzania wybranym obiektem.
- **Warunki końcowe:** Harmonogram zostaje zapisany w bazie danych ze statusem „Szkic” (niewidoczny dla pracowników).

- **Scenariusz:**

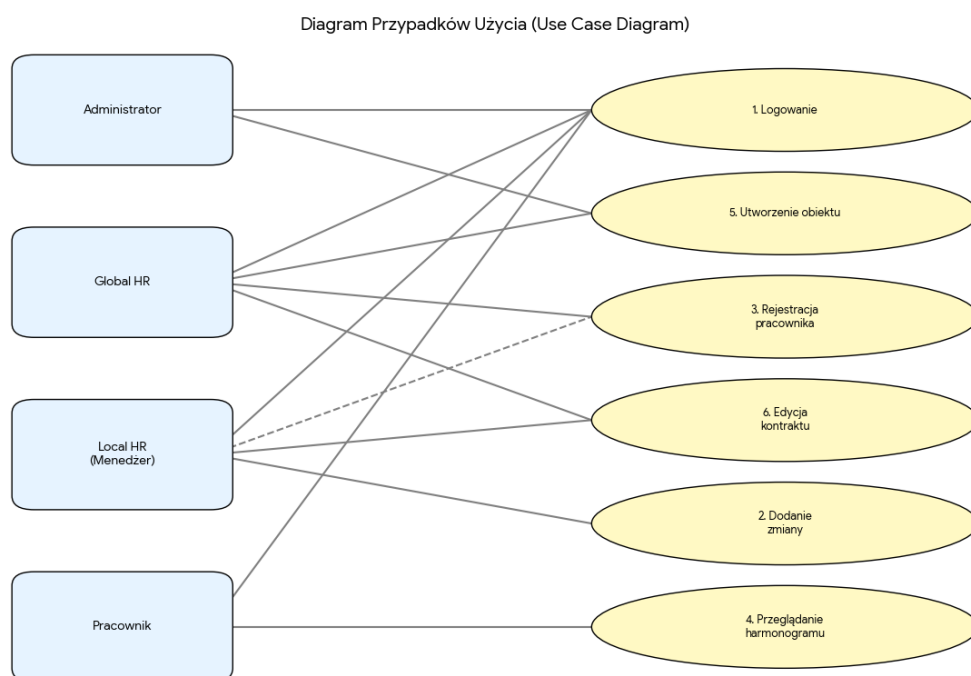
1. Użytkownik przechodzi do zakładki „Grafiki” (**Schedules**) i klika przycisk **Create Schedule**.
2. Użytkownik wybiera z listy obiekt oraz miesiąc, dla którego chce utworzyć grafik.
3. System generuje pustą siatkę harmonogramu (wiersze to pracownicy, kolumny to dni).
4. Użytkownik wprowadza symbole zmian (np. „R”, „N”) lub godziny (np. „08-16”) w komórki tabeli.
5. System na bieżąco waliduje wprowadzane dane (np. ostrzega przed naruszeniem 11-godzinnego odpoczynku dobowego).
6. Użytkownik klika przycisk **Save Changes** (Zapisz).
7. System przetwarza dane.
 - *Dane są poprawne:*
 - (a) System zapisuje harmonogram.
 - (b) System wyświetla komunikat „Schedule saved”.
 - *Wystąpił błąd zapisu:*
 - (a) System informuje o przyczynie błędu (np. brak połączenia).

Przypadek użycia: Publikacja harmonogramu (z weryfikacją konfliktów)

- **Aktor:** Local HR.
- **Cel:** Udostępnienie zaplanowanego grafiku pracownikom.
- **Warunki początkowe:** W systemie istnieje zapisany harmonogram o statusie „Szkic” (nieopublikowany).
- **Warunki końcowe:** Harmonogram zmienia status na „Opublikowany” i staje się widoczny na pulpitach pracowników.
- **Scenariusz:**
 1. Użytkownik znajduje na liście grafik oznaczony kolorem żółtym (Szkic).
 2. Użytkownik klika przycisk „Publikuj” (**Publish**).
 3. System przeprowadza weryfikację konfliktów (*Cross-check*): sprawdza, czy w tym samym okresie czasu nie istnieje inny opublikowany grafik zawierający tych samych pracowników.

4. System podejmuje decyzję:

- *Brak konfliktów:*
 - (a) System zmienia flagę `is_published` na `True`.
 - (b) Status grafiku zmienia się na zielony („Opublikowany”).
 - (c) Pracownicy otrzymują dostęp do grafiku w swoim panelu.
- *Wykryto konflikt (np. pracownik jest już w grafiku innego działu):*
 - (a) System blokuje publikację.
 - (b) System wyświetla komunikat błędu (**Error 409**): „Konflikt! Pracownicy powtarzają się w innym opublikowanym grafiku”.
 - (c) Grafik pozostaje w statusie „Szkic” do momentu rozwiązania konfliktu.



Rysunek 2.1: Diagram przypadków użycia, Źródło: Opracowanie własne

Rozdział 3

Narzędzia i technologie

Realizacja projektu „HRmonogram” wymagała doboru odpowiedniego stosu technologicznego, który zapewniłby skalowalność, bezpieczeństwo oraz wysoki komfort użytkowania (*User Experience*). Projekt został zrealizowany w architekturze *Single Page Application* (SPA), co zdeterminowało wybór narzędzi po stronie klienta.

3.1 Technologie warstwy klienckiej (Frontend)

Warstwa prezentacji odpowiada za interakcję z użytkownikiem oraz wizualizację danych pobieranych z serwera. Do jej budowy wykorzystano następujące technologie i biblioteki:

3.1.1 React.js

Jako fundament aplikacji wybrano bibliotekę React.js (w wersji 18.x), rozwijaną przez firmę Meta (Facebook). Jest to biblioteka języka JavaScript służąca do budowania interfejsów użytkownika.

React opiera się na architekturze komponentowej, co pozwala na budowanie skomplikowanych widoków z małych, niezależnych komponentów. Kluczowym atutem jest wykorzystanie wirtualnego modelu DOM (*Virtual DOM*), co minimalizuje liczbę operacji na rzeczywistym drzewie DOM przeglądarki, zapewniając wysoką wydajność aplikacji nawet przy częstych zmianach danych (np. podczas edycji grafiku).

3.1.2 JavaScript (ES6+)

Logika po stronie klienta została napisana w języku JavaScript, zgodnie ze standardem ECMAScript 2015+ (ES6). Wykorzystano nowoczesne funkcjonalności języka, takie jak funkcje strzałkowe, destrukuryzację obiektów, moduły oraz mechanizm `async/await` do

obsługi operacji asynchronicznych (komunikacja z API), co znacząco zwiększyło czytelność i zwiezłość kodu.

3.1.3 React Router DOM

Do obsługi nawigacji wewnątrz aplikacji SPA wykorzystano bibliotekę React Router. Standardowe aplikacje webowe przy każdej zmianie podstrony przeładowują całą witrynę. React Router umożliwia dynamiczną zmianę widoków bez odświeżania strony, symulując nawigację klasyczną, ale zachowując płynność aplikacji desktopowej. Umożliwia również definiowanie ścieżek chronionych, które blokują dostęp do widoków administracyjnych dla nieuprawnionych użytkowników.

3.1.4 HTML5 i CSS3

Warstwa strukturalna i wizualna oparta jest na standardach HTML5 oraz CSS3 (kaskadowe arkusze stylów). W projekcie zastosowano podejście *Responsive Web Design* (RWD), wykorzystując m.in. model *Flexbox*, aby aplikacja poprawnie wyświetlała się zarówno na monitorach komputerów, jak i urządzeniach mobilnych.

Node.js i NPM Choć Node.js jest środowiskiem serwerowym, w kontekście frontendu został wykorzystany jako środowisko uruchomieniowe dla narzędzi deweloperskich. Menedżer pakietów NPM (*Node Package Manager*) posłużył do instalacji bibliotek zewnętrznych oraz zarządzania zależnościami projektu.

3.1.5 Fetch API

Do komunikacji z serwerem (Backendem) wykorzystano natywny mechanizm przeglądarki Fetch API. Pozwala on na wysyłanie asynchronicznych żądań HTTP (GET, POST, PUT, DELETE) i obsługę odpowiedzi w formacie JSON, co stanowi standard w komunikacji z usługami typu RESTful.

3.1.6 Jwt-decode

Jest to lekka biblioteka służąca do dekodowania tokenów JWT (*JSON Web Tokens*) po stronie przeglądarki. Tokeny otrzymywane z serwera są zakodowane w formacie Base64Url. Biblioteka ta pozwala na odczytanie zawartości tokena bez konieczności wysyłania dodatkowego zapytania do serwera. Dzięki temu aplikacja frontendowa może natychmiastowo odczytać ID użytkownika oraz jego rolę (np. 'admin' lub 'employee') i na tej podstawie dynamicznie dostosować interfejs, realizując mechanizm kontroli dostępu (RBAC) po stronie klienta.

3.2 Technologie warstwy serwerowej

Fundamentem logiki biznesowej systemu „HRmonogram” jest język Python (wersji 3.x). Wybór ten podyktowany był czytelnością składni, co przyspiesza proces wytwórczy, oraz dostępnością rozbudowanego ekosystemu bibliotek wspierających tworzenie aplikacji internetowych. Poniżej scharakteryzowano kluczowe technologie i narzędzia wykorzystane w warstwie serwerowej.

3.2.1 Flask

Jako środowisko uruchomieniowe aplikacji wybrano Flask – lekki framework internetowy dla języka Python. Jest to rozwiązanie typu *micro-framework*, dostarcza ono jedynie niezbędnego minimum funkcjonalności, nie narzucając programiście sztywnej struktury projektu ani konkretnych rozwiązań bazodanowych.

Architektura Flaska opiera się na dwóch kluczowych filarach:

- **Zgodność ze standardem WSGI (*Web Server Gateway Interface*):** Gwarantuje to kompatybilność aplikacji z profesjonalnymi serwerami aplikacyjnymi i ułatwia przyszłe wdrożenie produkcyjne.
- **Biblioteka Werkzeug:** Jest to zestaw narzędzi, który wewnątrz Flaska odpowiada za niskopoziomową obsługę żądań HTTP, debugowanie oraz routing – mechanizm mapowania adresów URL na konkretne funkcje logiki biznesowej.

Dzięki swojej lekkości i elastyczności Flask okazał się idealnym narzędziem do budowy wydajnego interfejsu REST API, który w projekcie pełni rolę wyłącznego dostawcy danych dla frontendu.

3.2.2 Rozszerzenia frameworka Flask

Na potrzeby projektu framework Flask został rozbudowany o specjalistyczne biblioteki:

Flask-SQLAlchemy: Rozszerzenie integrujące aplikację z systemem mapowania obiektowo-relacyjnego (ORM). Umożliwia ono interakcję z bazą danych (SQLite) za pomocą obiektów Pythona, eliminując konieczność pisania surowych zapytań SQL i zwiększając bezpieczeństwo danych.

Flask-JWT-Extended: Biblioteka odpowiedzialna za implementację standardu *JSON Web Token* (JWT). Umożliwia tworzenie bezstanowego mechanizmu uwierzytelniania, generując i weryfikując podpisy cyfrowe tokenów przesyłanych w nagłówkach HTTP.

Flask-Bcrypt: Narzędzie kryptograficzne służące do jednokierunkowego haszowania haseł użytkowników algorytmem `bcrypt`. Zastosowanie soli (*salt*) chroni dane uwierzytelniające nawet w przypadku nieautoryzowanego dostępu do bazy danych.

Flask-CORS: Rozszerzenie obsługujące mechanizm *Cross-Origin Resource Sharing*. Jest ono niezbędne w architekturze rozproszonej, aby umożliwić bezpieczną komunikację między frontendem a backendem działającymi w różnych domenach lub na różnych portach.

3.2.3 Node.js i NPM

Node.js pełni kluczową rolę w procesie budowania aplikacji. Wykorzystano go jako środowisko uruchomieniowe JavaScript po stronie serwera/dewelopera, niezbędne do działania narzędzi budujących frontend.

Node Package Manager

Posłużył jako menedżer zależności, umożliwiając instalację i aktualizację bibliotek zewnętrznych (takich jak React czy pakiety testowe) oraz uruchamianie skryptów automatyzujących proces kompilacji wersji produkcyjnej systemu.

3.3 Warstwa bazy danych

Jako system zarządzania bazą danych (DBMS) w projekcie wykorzystano SQLite. Jest to silnik relacyjny, który w odróżnieniu od tradycyjnych rozwiązań typu klient-serwer (takich jak MySQL czy PostgreSQL), charakteryzuje się architekturą wbudowaną (*embedded*).

Wybór tego rozwiązania podyktowany był specyfiką projektu oraz następującymi zaletami technologicznymi:

3.3.1 Architektura bezserwerowa (Serverless)

SQLite nie wymaga uruchamiania osobnego procesu w tle ani konfiguracji połączeń sieciowych. Cały silnik bazy danych jest zintegrowany bezpośrednio z aplikacją, co eliminuje opóźnienia sieciowe i znacząco upraszcza proces wdrażania oprogramowania.

3.3.2 Przenośność (Portability)

Cała baza danych – w tym definicje tabel, indeksy oraz rekordy – przechowywana jest w pojedynczym pliku. Umożliwia to łatwe tworzenie kopii zapasowych poprzez proste sko-

piowanie pliku oraz bezproblemowe przenoszenie projektu między różnymi środowiskami deweloperskimi.

3.3.3 Pełna obsługa standardu SQL i ACID

Mimo swojej lekkości, SQLite jest pełnoprawną bazą relacyjną. Obsługuje większość standardu SQL-92 oraz gwarantuje transakcyjność (ACID). Oznacza to, że operacje na danych (np. dodawanie nowego grafiku) są atomowe – albo wykonają się w całości, albo wcale, co chroni system przed niespójnością danych w przypadku awarii zasilania lub błędu aplikacji.

3.3.4 Wydajność w skali projektu

Dla aplikacji klasy MVP (*Minimum Viable Product*) oraz systemów o małym i średnim natężeniu ruchu (kilkuset użytkowników), SQLite oferuje wydajność porównywalną lub wyższą od dużych serwerów bazodanowych, ze względu na brak narzutu komunikacji sieciowej.

W kontekście projektu inżynierskiego użycie SQLite pozwoliło skupić się na implementacji złożonej logiki biznesowej aplikacji, zamiast na administracji serwerem bazodanowym.

3.4 Narzędzia wspomagające proces wytwórczy

Oprócz technologii stanowiących składowe systemu, w procesie realizacji projektu wykorzystano szereg narzędzi wspierających pisanie kodu, zarządzanie wersjami oraz testowanie.

3.4.1 Środowisko programistyczne (IDE) – Visual Studio Code

Jako główny edytor kodu wybrano Visual Studio Code. Decyzja ta podyktowana była jego lekkością oraz bogatym ekosystemem rozszerzeń wspierających zarówno rozwój backendu (Python/Flask), jak i frontendu (React/JSX). Wykorzystano wbudowany terminal do obsługi serwera oraz narzędzia do formatowania kodu (Prettier, Black), co pozwoliło na zachowanie spójnego stylu w całym projekcie.

3.4.2 System kontroli wersji – Git i GitHub

Do zarządzania kodem źródłowym wykorzystano rozproszony system kontroli wersji Git. Umożliwiło to śledzenie historii zmian, bezpieczne eksperymentowanie na osobnych gałęziach (*branches*) oraz tworzenie kopii zapasowej projektu. Zdalne repozytorium zostało

osadzone w serwisie GitHub, co zapewnia bezpieczeństwo danych i dostęp do kodu z różnych urządzeń.

Rozdział 4

Implementacja

4.1 Technologie i narzędzia

Projekt systemu „HRmonogram” zrealizowano w oparciu o architekturę wielowarstwową, w której warstwa dostępu do danych, logika biznesowa oraz interfejs użytkownika stanowią odrębne, współpracujące ze sobą moduły. Poniżej scharakteryzowano środowisko technologiczne wykorzystane do implementacji warstwy serwerowej (*backend*) oraz bazodanowej.

4.1.1 Baza danych

W projekcie zastosowano relacyjny system zarządzania bazą danych SQLite. Decyzja o wyborze tego rozwiązania podyktowana była jego bezserwerową architekturą, co znacząco upraszcza proces wdrażania aplikacji i konfiguracji środowiska deweloperskiego. Baza danych przechowywana jest w pojedynczym pliku, co zapewnia wysoką przenośność projektu bez utraty spójności danych.

Do interakcji z bazą danych wykorzystano bibliotekę SQLAlchemy – zaawansowany system mapowania obiektowo-relacyjnego (ORM) dla języka Python. Zastosowanie warstwy abstrakcji ORM pozwoliło na:

- Definiowanie schematu bazy danych w postaci klas języka Python (modeli), co zwiększa czytelność kodu i ułatwia zarządzanie migracjami.
- Uniezależnienie logiki aplikacji od konkretnego silnika bazodanowego (możliwość łatwej migracji np. na PostgreSQL w przyszłości).
- Zwiększenie bezpieczeństwa poprzez automatyczną parametryzację zapytań SQL, eliminującą ryzyko ataków typu *SQL Injection*.

Zaimplementowany model danych obejmuje następujące kluczowe encje:

User: Reprezentacja użytkowników systemu wraz z informacjami o rolach (RBAC) oraz danymi uwierzytelniającymi.

Object i Department: Encje odwzorowujące hierarchiczną strukturę organizacyjną przedsiębiorstwa.

WorkSchedule: Encja przechowująca harmonogram pracy dla danego obiektu w określonym przedziale czasowym, zawierająca status publikacji.

Schedule: Szczegółowe rekordy zmian roboczych przypisane do konkretnych pracowników.

ShiftPreference/Template: Tabele słownikowe przechowujące konfigurację walidacji czasu pracy oraz definicje szablonów zmian (legenda).

4.1.2 Backend

Warstwa serwerowa aplikacji została zrealizowana w języku Python przy użyciu mikroframeworka Flask. Wybór tego narzędzia uzasadniony jest jego elastycznością oraz brakiem narzuconych z góry wzorców, co pozwoliło na dostosowanie struktury projektu do specyficznych wymagań systemu harmonogramowania.

Architekturę backendu oparto na wzorcu *Blueprints*, co umożliwiło modularny podział aplikacji na niezależne komponenty funkcjonalne, takie jak: **auth** (uwierzytelnianie), **schedules** (zarządzanie grafikami) czy **stats** (zbiór danych statystycznych). Takie podejście ułatwia konserwację kodu i pracę zespołową.

W implementacji wykorzystano następujące kluczowe rozszerzenia frameworka Flask:

Flask-JWT-Extended: Odpowiada za realizację bezstanowego mechanizmu autoryzacji opartego na standardzie JSON Web Tokens (JWT). Każde żądanie HTTP do chronionych zasobów API wymaga dołączenia ważnego tokena, co zapewnia bezpieczeństwo komunikacji i identyfikację użytkownika.

Flask-Bcrypt: Biblioteka implementująca funkcję skrótu bcrypt, wykorzystywana do bezpiecznego haszowania haseł użytkowników. Zastosowanie soli (*salt*) oraz adaptacyjnego kosztu obliczeniowego skutecznie chroni dane uwierzytelniające przed atakami typu *brute-force* i *rainbow tables*.

Flask-CORS: Moduł obsługujący mechanizm *Cross-Origin Resource Sharing*, niezbędny do umożliwienia bezpiecznej wymiany danych pomiędzy klientem (Frontend) a serwerem API, działającymi w różnych domenach lub na różnych portach lokalnych.

Komunikacja z warstwą prezentacji odbywa się poprzez interfejs REST API, który udostępnia zasoby w formacie JSON, realizując operacje zgodnie z metodami protokołu HTTP (GET, POST, PUT, DELETE).

4.1.3 Frontend

Interfejs użytkownika systemu „HRmonogram” został zrealizowany w modelu SPA (*Single Page Application*), co zapewnia płynność działania zbliżoną do aplikacji desktopowych poprzez dynamiczne odświeżanie treści bez konieczności przeładowywania całej strony.

Fundamentem technologicznym warstwy prezentacji jest biblioteka React.js. Wybór tej technologii podyktowany był jej komponentową architekturą, która umożliwia budowanie złożonych interfejsów z mniejszych, reużywalnych elementów (komponentów), co znacząco przyspiesza proces wytwórczy i ułatwia utrzymanie kodu.

W implementacji frontendu wykorzystano następujące kluczowe biblioteki i mechanizmy:

React Router DOM: Biblioteka odpowiedzialna za obsługę routingu po stronie klienta.

Umożliwia ona mapowanie adresów URL na konkretne widoki aplikacji (np. `/login`, `/dashboard`) oraz zabezpieczanie dostępu do ścieżek dla niezalogowanych użytkowników.

React Hooks: Zastosowano nowoczesne podejście do zarządzania stanem aplikacji i cyklem życia komponentów, wykorzystując wbudowane funkcje (*Hooks*), takie jak `useState` (zarządzanie danymi lokalnymi) oraz `useEffect` (obsługa efektów ubocznych, np. pobieranie danych).

Fetch API: Do asynchronicznej komunikacji z serwerem Backend wykorzystano natywny mechanizm przeglądarki `fetch`. Umożliwia on wysyłanie żądań HTTP i odbieranie odpowiedzi w formacie JSON, co stanowi standard w komunikacji z REST API.

CSS / Inline Styles: Warstwa wizualna została zaimplementowana przy użyciu standardowych arkuszy stylów CSS oraz stylowania bezpośredniego (*inline styles*), co pozwoliło na szybkie prototypowanie i pełną kontrolę nad wyglądem elementów interfejsu.

Całość aplikacji frontendowej jest budowana i optymalizowana przy użyciu narzędzi zawartych w środowisku Node.js oraz menedżera pakietów `npm`, który zarządza wszystkimi zależnościami projektu.

4.2 Struktura projektu

Przejrzysta organizacja kodu źródłowego jest kluczowa dla łatwości utrzymania i dalszego rozwoju oprogramowania. Projekt „HRmonogram” został podzielony na dwa niezależne katalogi główne: **backend** (logika serwerowa) oraz **frontend** (interfejs użytkownika). Poniżej omówiono szczegółowo strukturę części serwerowej.

4.2.1 Struktura projektu backendu (Flask)

Część serwerowa aplikacji została zaprojektowana w oparciu o modułarną strukturę pakietów. Główna instancja aplikacji jest inicjalizowana jako obiekt globalny w pliku startowym, co upraszcza konfigurację i dostęp do współdzielonych zasobów (takich jak baza danych).

Aby zapewnić skalowalność i czytelność kodu, logika biznesowa została podzielona na niezależne moduły funkcjonalne z wykorzystaniem mechanizmu Flask Blueprints.

Drzewo katalogów projektu backendowego prezentuje się następująco:

```
backend/  
  app/  
    __init__.py  
    models.py  
    routes/  
      auth.py  
      users.py  
      objects.py  
      schedules.py  
      stats.py  
  requirements.txt  
  run.py  
  create_admin.py
```

Rysunek 4.1: Struktura projektu backendu. Źródło: Opracowanie własne

Opis kluczowych elementów struktury:

1. Punkt wejścia (`run.py`):

Plik ten pełni rolę startera. Jego jedynym zadaniem jest zaimportowanie funkcji tworzącej aplikację z pakietu `app` i uruchomienie serwera deweloperskiego. Oddzielenie logiki uruchomieniowej od definicji aplikacji jest dobrą praktyką inżynierską.

2. Inicjalizacja (`app/__init__.py`):

To serce aplikacji backendowej. W tym pliku:

- Inicjalizowana jest instancja klasy `Flask`.
- Konfigurowane są parametry połączenia z bazą danych oraz klucze szyfrujące.
- Inicjalizowane są globalne rozszerzenia: `SQLAlchemy` (ORM), `Bcrypt` (haszowanie), `JWTManager` (autoryzacja) oraz `CORS`.
- Rejestrowane są tzw. *Blueprints* (Schematy), które importują ścieżki (*routes*) z katalogu `app/routes/`.

3. Warstwa modeli (`app/models.py`):

Plik ten zawiera definicje klas reprezentujących tabele w bazie danych. Każda klasa

dziedziczy po `db.Model`, a jej atrybuty odpowiadają kolumnom w bazie. Modele definiują również relacje między tabelami (np. `relationship` w SQLAlchemy), co pozwala na łatwe nawigowanie po powiązanych danych (np. pobranie wszystkich zmian przypisanych do konkretnego grafiku).

4. Warstwa kontrolerów i logiki biznesowej (`app/routes/`):

Zastosowanie mechanizmu *Blueprints* pozwoliło na logiczny podział API na mniejsze, tematyczne moduły. Każdy plik w tym katalogu pełni rolę kontrolera, mapując żądania HTTP (GET, POST, PUT, DELETE) na konkretne funkcje Pythona. Poniżej przedstawiono szczegółową charakterystykę poszczególnych plików:

`auth.py` (Uwierzytelnianie): Odpowiada za proces logowania do systemu. Zawiera endpoint `/login`, który przyjmuje poświadczenia użytkownika (e-mail, hasło), weryfikuje ich poprawność przy użyciu `bcrypt`, a w przypadku sukcesu generuje i zwraca token dostępu (*Access Token*) biblioteki `flask-jwt-extended`.

`users.py` (Zarządzanie pracownikami): Implementuje pełny zestaw operacji CRUD (*Create, Read, Update, Delete*) dla encji Użytkownika. Moduł ten obsługuje:

- Rejestrację nowych pracowników z automatyczną walidacją unikalności adresu e-mail.
- Filtrowanie i sortowanie listy pracowników (np. po nazwisku, obiekcie czy dziale).
- Edycję danych kadrowych oraz przypisywanie ról w systemie (Admin, HR, Employee).

`objects.py` (Struktura organizacyjna): Kontroler zarządzający fizyczną strukturą przedsiębiorstwa. Umożliwia dodawanie nowych obiektów (np. hoteli, restauracji) oraz definiowanie działów wewnątrz nich. Dane te są wykorzystywane później do grupowania pracowników i przypisywania grafików.

`schedules.py` (Harmonogramowanie): Centralny i najbardziej złożony moduł systemu. Odpowiada za:

- Tworzenie i edycję nagłówków grafików (`WorkSchedule`) oraz poszczególnych zmian (`Schedule`).
- Obsługę mechanizmu „Szkic” vs „Opublikowany”.
- Logikę wykrywania konfliktów: Przed publikacją grafiku uruchamiana jest procedura sprawdzająca, czy pracownicy nie zostali przypisani do innego harmonogramu w tym samym przedziale czasowym.

`stats.py` (Analityka i Dashboard): Moduł agregujący dane dla pulpitu menedżerskiego. Zamiast zwracać surowe rekordy, wykonuje zapytania analityczne,

obliczając wskaźniki KPI (*Key Performance Indicators*), takie jak: liczba aktywnych grafików, statusy uzupełnienia harmonogramów dla poszczególnych obiektów czy liczba aktywnych pracowników.

contracts.py (Zarządzanie kontraktami): Moduł pomocniczy umożliwiający definiowanie i edycję parametrów zatrudnienia, takich jak typ umowy czy wymiar etatu, które są niezbędne do prawidłowego planowania czasu pracy.

labor_code_validator.py (Walidator Kodeksu Pracy): Specjalistyczny moduł pomocniczy, niebędący bezpośrednio endpointem API, lecz biblioteką funkcji wykorzystywaną przez `schedules.py`. Implementuje algorytmy sprawdzające zgodność planu z regułami prawa pracy, m.in. weryfikację 11-godzinnego nieprzerwanego odpoczynku dobowego pomiędzy zmianami.

Taka separacja odpowiedzialności sprawia, że kod jest czytelny, łatwy w testowaniu i zgodny z zasadą pojedynczej odpowiedzialności (*Single Responsibility Principle*).

4.2.2 Struktura projektu frontend (React)

Warstwa prezentacji została wygenerowana przy użyciu standardowego szablonu dla aplikacji React, co zapewnia zgodność z najlepszymi praktykami branżowymi. Kod źródłowy zorganizowano w sposób modułowy, separując logikę sterującą, widoki oraz zasoby statyczne.

Poniżej przedstawiono strukturę katalogów projektu klienckiego:

Opis kluczowych elementów struktury:

1. Punkt wejścia (`public/index.html` oraz `src/index.js`):

Ponieważ aplikacja działa w modelu SPA, plik `index.html` zawiera tylko jeden kontener `div` o identyfikatorze `root`. Plik `src/index.js` jest odpowiedzialny za „wstrzyknięcie” całej aplikacji React do tego kontenera oraz inicjalizację mechanizmów globalnych (np. trybu ścisłego `React.StrictMode`).

2. Główny węzeł aplikacji (`src/App.js`):

Jest to najważniejszy komponent sterujący. W tym pliku zdefiniowano:

- **Routing:** Mapowanie adresów URL (np. `/employees`, `/schedules`) na konkretne komponenty przy użyciu biblioteki `react-router-dom`.
- **Kontekst autoryzacji:** Logikę sprawdzającą, czy użytkownik posiada token JWT oraz odpowiednią rolę (Admin/Employee) przed wyświetleniem żądanego widoku.
- **Layout:** Wspólny układ strony (np. pasek nawigacyjny `Header`), który renderowany jest na każdym podwidoku.

```
frontend/  
  public/  
    index.html  
    manifest.json  
    logo.png  
  src/  
    components/  
      Login.js  
      Dashboard.js  
      EmployeeDashboard.js  
      Schedules.js  
      CreateNewSchedule.js  
      Employees.js  
    authUtils.js  
    App.js  
    index.js  
    App.css  
  package.json  
  package-lock.json
```

Rysunek 4.2: Struktura projektu frontend (React), Źródło: Opracowanie własne.

3. Katalog komponentów (`src/components/`):

Zawiera on wszystkie widoki i elementy interfejsu. W projekcie przyjęto podział funkcjonalny, gdzie każdy plik JavaScript (np. `Login.js`) zawiera zarówno logikę (obsługa zdarzeń, pobieranie danych), jak i warstwę prezentacji (JSX). Najważniejsze grupy komponentów to:

- **Administracyjne:** `CreateUser.js`, `ManageObjects.js` – dostępne tylko dla ról Global HR/Admin.
- **Operacyjne:** `CreateNewSchedule.js` – zaawansowany komponent zawierający logikę interaktywnej tabeli do układania grafiku.
- **Informacyjne:** `EmployeeDashboard.js` – widok „read-only” prezentujący grafik z perspektywy pracownika.

4. Logika pomocnicza (`src/authUtils.js`):

Wydzielony moduł zawierający funkcje narzędziowe, np. `getUserRole(token)`, która dekoduje ciąg Base64 tokena JWT, aby wydobyć z niego informacje o uprawnieniach zalogowanego użytkownika bez konieczności ponownego pytania serwera.

5. Zarządzanie zależnościami (`package.json`):

Plik ten definiuje listę bibliotek zewnętrznych wykorzystanych w projekcie (m.in.

`react`, `react-router-dom`, `jwt-decode`) oraz skrypty służące do uruchamiania środowiska deweloperskiego (`npm start`) i budowania wersji produkcyjnej (`npm build`).

4.3 Implementacja warstwy danych

System „HRmonogram” opiera się na relacyjnym modelu danych (RDBMS). Ze względu na charakterystykę projektu, wybrano silnik SQLite, który w środowisku produkcyjnym aplikacji o małej i średniej skali zapewnia wystarczającą wydajność, oferując jednocześnie pełną obsługę transakcyjności (ACID) oraz standardu SQL. Interakcja z bazą danych nie odbywa się poprzez bezpośrednie zapytania SQL, lecz za pośrednictwem biblioteki SQLAlchemy.

4.3.1 Modele danych i kluczowe encje

Struktura bazy danych została znormalizowana do trzeciej postaci normalnej (3NF), aby uniknąć powtarzania danych i zapewnić ich spójność. Poniżej przedstawiono implementację kluczowych modeli systemu.

1. Model Użytkownika (User)

Jest to centralna encja systemu, reprezentująca każdego aktora (od Administratora po Pracownika). Model ten zawiera nie tylko dane osobowe i uwierzytelniające, ale także klucze obce (*ForeignKey*) wiążące użytkownika ze strukturą organizacyjną firmy.

Warto zwrócić uwagę na implementację metody `set_password`, która wykorzystuje bibliotekę `bcrypt` do haszowania hasła przed przypisaniem go do atrybutu modelu.

2. Struktura Organizacyjna (Object)

Model ten reprezentuje fizyczne placówki przedsiębiorstwa (np. hotele, restauracje). Zastosowano tu relację typu Jeden-do-Wielu (*One-to-Many*), gdzie jeden obiekt może posiadać wielu przypisanych pracowników oraz wiele historycznych grafików pracy.

3. Harmonogramy (WorkSchedule i Schedule)

Aby zoptymalizować zarządzanie czasem pracy, system rozdziela pojęcie „Grafiku” na dwie powiązane encje:

- **WorkSchedule (Nagłówek):** Przechowuje metadane grafiku dla danego obiektu i miesiąca (np. `start_date`, `end_date`) oraz flagę `is_published`. W modelu tym zastosowano również wzorzec *Snapshot* – pole `shift_preference_snapshot` przechowuje w formacie JSON kopię ustawień i legendy skrótów obowiązującą

```

class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)
    name = db.Column(db.String(100), nullable=False)
    role = db.Column(db.String(20), nullable=False)

    # Klucze obce
    object_id = db.Column(db.Integer, db.ForeignKey('objects.id'), nullable=True)
    department_id = db.Column(db.Integer, db.ForeignKey('departments.id'), nullable=True)
    contract_id = db.Column(db.Integer, db.ForeignKey('contracts.id'), nullable=True)

    must_change_password = db.Column(db.Boolean, default=True)

    # Relacje
    object = db.relationship('Object', backref=db.backref('users', lazy=True))
    department = db.relationship('Department', backref=db.backref('users', lazy=True))
    contract = db.relationship('Contract', backref=db.backref('users', lazy=True))

```

Rysunek 4.3: Definicja modelu User (plik backend/app/models.py), Źródło: Opracowanie własne

w momencie tworzenia grafiku. Zabezpiecza to dane historyczne przed zmianami w globalnej konfiguracji obiektu.

- **Schedule (Szczegóły):** Przechowuje konkretne zmiany pracownicze. Każdy rekord to przypisanie pracownika do konkretnego dnia i typu zmiany (np. „08:00-16:00”).

```

class Schedule(db.Model):
    __tablename__ = 'schedules'
    id = db.Column(db.Integer, primary_key=True)
    employee_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    object_id = db.Column(db.Integer, db.ForeignKey('objects.id'), nullable=True)

    work_schedule_id = db.Column(db.Integer, db.ForeignKey('work_schedules.id'), nullable=True)

    date = db.Column(db.Date, nullable=False)
    shift = db.Column(db.String(20), nullable=False)

    employee = db.relationship('User', backref=db.backref('schedules', lazy=True))
    object = db.relationship('Object', backref=db.backref('daily_shifts', lazy=True))

```

Rysunek 4.4: Definicja modelu pojedynczej zmiany (plik backend/app/models.py), Źródło: Opracowanie własne

Zastosowana struktura pozwala na szybkie filtrowanie zmian zarówno po stronie obiektu (widok Managera), jak i po stronie użytkownika (widok „Mój Grafik”), wykorzystując

wbudowane w SQLAlchemy mechanizmy *lazy loading* oraz *backreferences*.

4.3.2 Relacje między tabelami

Relacje między tabelami w bazie danych zostały zdefiniowane zgodnie z zasadami integralności referencyjnej, co zapewnia spójność danych:

- Tabela `objects` pełni rolę nadrzędną dla tabeli `users`, tworząc relację jeden-do-wielu. Umożliwia to przypisanie wielu pracowników do jednej fizycznej placówki, przy czym każdy pracownik należy do jednego obiektu.
- Tabela `objects` jest również powiązana relacją jeden-do-wielu z tabelą `work_schedules`. Pozwala to na przechowywanie wielu miesięcznych harmonogramów w ramach jednego oddziału firmy.
- Tabela `work_schedules` stanowi punkt odniesienia dla tabeli `schedules` (szczegóły zmian). Relacja ta została skonfigurowana z mechanizmem kaskadowego usuwania, co gwarantuje, że usunięcie grafiku głównego automatycznie usuwa wszystkie powiązane z nim zmiany szczegółowe.
- Tabela `users` posiada relację jeden-do-wielu z tabelą `schedules`, co pozwala na przypisanie konkretnych zmian roboczych do danego pracownika oraz efektywne filtrowanie grafiku z perspektywy użytkownika.

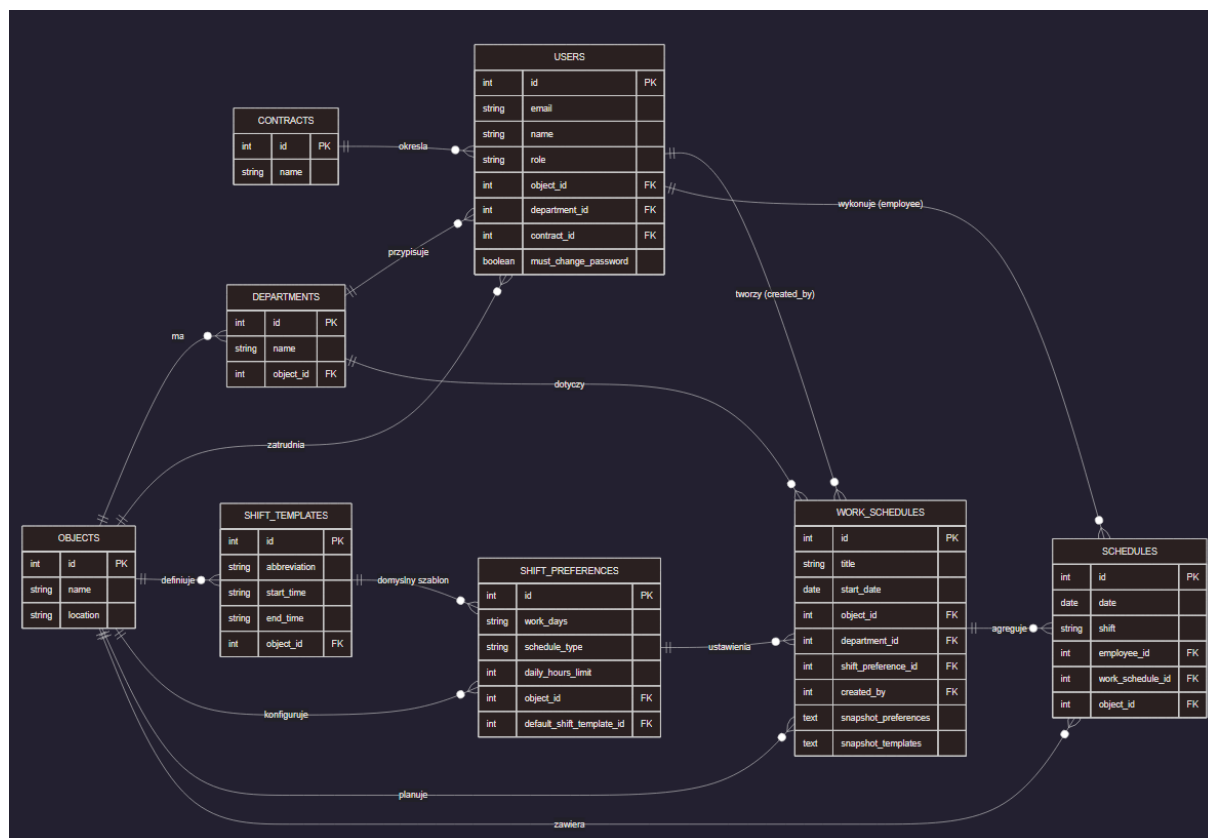
4.4 Obsługa routingu i interfejs API

Komunikacja między warstwą prezentacji (*Frontend*) a logiką biznesową (*Backend*) odbywa się w oparciu o architekturę REST (*Representational State Transfer*). Serwer udostępnia zestaw punktów końcowych (*endpoints*), które realizują operacje na zasobach przy użyciu standardowych metod protokołu HTTP.

4.4.1 Organizacja routingu (Blueprints)

W celu zachowania modułowości systemu i czytelności kodu, w projekcie wykorzystano mechanizm *Flask Blueprints*. Pozwala on na podział aplikacji na mniejsze, niezależne komponenty, które posiadają własne definicje ścieżek, a następnie są rejestrowane w głównej instancji aplikacji.

Dzięki temu podejściu logika dotycząca pracowników nie miesza się z logiką harmonogramów czy uwierzytelniania. Rejestracja poszczególnych modułów odbywa się w pliku inicjalizacyjnym `__init__.py`:



Rysunek 4.5: Diagram ERD projektu bazy danych, Źródło: Opracowanie własne

```

from app.routes.auth import auth_bp
from app.routes.users import users_bp
from app.routes.objects import objects_bp
from app.routes.schedules import schedules_bp
from app.routes.stats import stats_bp

app.register_blueprint(auth_bp, url_prefix='/auth')
app.register_blueprint(users_bp, url_prefix='/users')
app.register_blueprint(objects_bp, url_prefix='/objects')
app.register_blueprint(schedules_bp, url_prefix='/schedules')
app.register_blueprint(stats_bp, url_prefix='/stats')

```

Rysunek 4.6: Moduły routingu w aplikacji Flask. Źródło: Opracowanie własne

4.4.2 Zestawienie punktów końcowych (Endpoints)

System udostępnia szeroki interfejs pokrywający wymagania funkcjonalne. Poniższa tabela (4.1) prezentuje kluczowe endpointy zaimplementowane w projekcie.

4.4.3 Format wymiany danych i kody statusu

Wymiana danych odbywa się w formacie JSON, co zapewnia łatwość przepływu i parsowania po stronie klienta (React). Każda odpowiedź serwera zawiera standardowy

Metoda	Ścieżka	Opis funkcjonalności	Wymagana Rola
POST	/auth/login	Weryfikacja poświadczeń i generowanie tokena JWT.	(Publiczny)
GET	/users/employees	Pobranie listy pracowników (z możliwością filtrowania).	Admin, HR
POST	/users/create	Rejestracja nowego użytkownika w systemie.	Admin, Global HR
PUT	/users/<id>	Aktualizacja danych pracownika (kontrakt, obiekt).	Admin, HR
GET	/objects	Pobranie struktury firmy.	Zalogowany
GET	/schedules	Pobranie listy grafików.	Zalogowany
POST	/schedules/create	Utworzenie nowego harmonogramu z walidacją zmian.	Admin, Local HR
POST	/schedules/<id>/publish	Publikacja grafiku, uruchamia algorytm wykrywania konfliktów.	Local HR
GET	/schedules/user/<id>	Pobranie zmian dla konkretnego pracownika (widok "Mój Grafik").	Zalogowany
GET	/stats/dashboard	Pobranie statystyk KPI.	Admin, Global HR

Tabela 4.1: Wykaz kluczowych końcówek API systemu HRmonogram, Źródło: Opracowanie własne.

kod statusu HTTP, informujący o rezultacie operacji:

200 / 201 Created

Operacja zakończona sukcesem (np. pomyślne zapisanie grafiku).

400 Bad Request

Błąd walidacji danych wejściowych (np. niepoprawny format e-maila).

401 Unauthorized

Brak ważnego tokena JWT (próba dostępu bez logowania).

403 Forbidden

Próba wykonania akcji powyżej swoich uprawnień (np. Pracownik próbuje usunąć Obiekt).

409 Conflict

Naruszenie integralności danych (np. próba rejestracji na zajęty e-mail lub publikacja grafiku z konfliktami czasowymi).

500 Internal Server Error

Nieoczekiwany błąd po stronie serwera.

4.5 Mechanizmy bezpieczeństwa i autoryzacji

Ze względu na charakter przetwarzanych danych (dane osobowe, ewidencja czasu pracy), bezpieczeństwo systemu „HRmonogram” było priorytetem podczas implementacji. Zastosowano wielowarstwowe podejście do ochrony zasobów.

1. Uwierzytelnianie bezstanowe (*Stateless Authentication*)

System nie wykorzystuje klasycznych sesji serwerowych (*Cookies*), lecz opiera się na standardzie JWT (*JSON Web Token*).

- Po poprawnym zalogowaniu serwer generuje podpisany cyfrowo token, zawierający `user_id` oraz `role`.
- Token ten jest przesyłany przez frontend w nagłówku `Authorization: Bearer <token>` przy każdym zapytaniu.
- Token posiada krótki czas ważności (np. 1 godzina), co minimalizuje ryzyko w przypadku jego przechwycenia.

2. Ochrona haseł

Hasła użytkowników nigdy nie są przechowywane w bazie danych w formie jawnej. Wykorzystano bibliotekę `Bcrypt`, która stosuje algorytm haszowania z solą (*salting*). Nawet w przypadku wycieku bazy danych odtworzenie oryginalnych haseł jest obliczeniowo nieopłacalne.

3. Kontrola dostępu oparta na rolach (RBAC)

Wdrożono system dekoratorów w Pythonie (np. `@role_required`), które działają jako warstwa pośrednia. Przed wykonaniem funkcji biznesowej (np. usunięcie grafiku), system sprawdza, czy rola zaszyfrowana w tokenie użytkownika uprawnia do tej operacji. Próba nieautoryzowanego dostępu kończy się błędem 403 `Forbidden`.

4.6 Kluczowe algorytmy

Poza standardowymi operacjami CRUD, w systemie zaimplementowano autorskie algorytmy zapewniające spójność i poprawność planowania czasu pracy.

4.6.1 Walidator Kodeksu Pracy

W celu zapewnienia zgodności planowanych grafików z obowiązującymi przepisami prawa, zaimplementowano klasę pomocniczą `LaborCodeValidator`. Moduł ten działa w czasie rzeczywistym podczas edycji harmonogramu, analizując ciągłość zmian pracownika i zgłaszając ewentualne naruszenia w trzech kluczowych obszarach:

Weryfikacja nieprzerwanego odpoczynku dobowego:

Algorytm sortuje zmiany pracownika chronologicznie i oblicza różnicę czasu pomiędzy godziną zakończenia jednej zmiany a godziną rozpoczęcia następnej. Jeśli wyliczona przerwa wynosi mniej niż 11 godzin, system flaguje to jako naruszenie przepisów BHP i Kodeksu Pracy, ostrzegając menedżera przed zapisaniem błędnego grafiku.

Kontrola tygodniowego wymiaru czasu pracy:

System zlicza sumę godzin przepracowanych przez pracownika w danym tygodniu rozliczeniowym. Algorytm sprawdza, czy suma ta nie przekracza ustawowej normy (standardowo 40 godzin, z uwzględnieniem ewentualnych nadgodzin). Przekroczenie zdefiniowanego limitu skutkuje wyświetleniem komunikatu o konieczności redukcji godzin lub zaplanowania dnia wolnego.

Reguła wolnych niedziel:

Zaimplementowano logikę sprawdzającą częstotliwość pracy w niedziele. Mechanizm weryfikuje, czy pracownik ma zapewnioną wymaganą liczbę niedziel wolnych od pracy (co najmniej jedną na 4 tygodnie), zapobiegając sytuacjom permanentnej pracy w dni ustawowo wolne.

Dzięki automatyzacji tych procesów aplikacja zdejmuje z menedżera (Local HR) obowiązek ręcznego przeliczania godzin i przerw, minimalizując ryzyko błędów prawnych.

4.6.2 Algorytm wykrywania konfliktów czasowych

Podczas publikacji grafiku system uruchamia procedurę weryfikacyjną, która zapobiega podwójnemu przypisaniu pracownika.

Algorytm pobiera listę pracowników z publikowanego grafiku i przeszukuje bazę danych pod kątem innych, aktywnych i opublikowanych grafików w tym samym przedziale dat. Jeśli system wykryje, że pracownik Jan Kowalski ma już zaplanowane zmiany w innym grafiku w tym samym czasie, publikacja jest blokowana, a menedżer otrzymuje informację o kolizji.

Rozdział 5

Testowanie

ten rozdział poświęcono procesowi weryfikacji i walidacji wytworzonego oprogramowania. Jego celem jest potwierdzenie zgodności systemu „HRmonogram” z założeniami projektowymi, a także wykazanie stabilności i bezpieczeństwa zaimplementowanych rozwiązań. Testowanie oprogramowania jest kluczowym etapem cyklu wytwórczego, pozwalającym na wczesne wykrycie błędów oraz zapewnienie wysokiej jakości kodu końcowego.

5.1 Środowisko i strategia testów

Do weryfikacji poprawności działania warstwy serwerowej wykorzystano framework **pytest**. Wybór ten podyktowany był jego zwięzłą składnią oraz zaawansowanym mechanizmem *fixtures*, który pozwala na efektywne przygotowywanie środowiska testowego (np. tymczasowej bazy danych) przed każdym testem.

Testy zostały podzielone na dwie główne kategorie:

1. **Testy jednostkowe (*Unit Tests*):** Weryfikujące izolowane fragmenty logiki biznesowej (modele, algorytmy) bez angażowania całej infrastruktury serwera.
2. **Testy integracyjne (*Integration Tests*):** Sprawdzające poprawność działania punktów końcowych (*endpoints*) API oraz interakcję z bazą danych.

5.2 Testy jednostkowe (Unit Testing)

Celem testów jednostkowych było sprawdzenie kluczowych mechanizmów aplikacji w izolacji. Skupiono się tutaj na modelach danych oraz algorytmach walidacji czasu pracy.

Rysunek 5.1: Weryfikacja modelu Użytkownika i haszowania haseł

Poniższy test sprawdza, czy mechanizm `bcrypt` poprawnie szyfruje hasło i czy metoda weryfikująca odrzuca błędne poświadczenia.

```
from app.models import User

def test_new_user_password_hashing():
    """
    Test sprawdza, czy ustawienie hasła generuje hash
    i czy weryfikacja działa poprawnie.
    """

    user = User(email="test@hrmonogram.pl", name="Jan Testowy")

    user.set_password("TajneHaslo123")

    assert user.password_hash is not None
    assert user.password_hash != "TajneHaslo123"
    assert user.check_password("TajneHaslo123") is True
    assert user.check_password("ZleHaslo") is False
```

Rysunek 5.1: Test jednostkowy modelu User, Źródło: Opracowanie własne.

Rysunek 5.2: Test algorytmu Walidatora Kodeksu Pracy

Kluczowym elementem logiki jest sprawdzanie 11-godzinnego odpoczynku. Testujemy tu samą klasę `LaborCodeValidator` bez uruchamiania serwera.

5.3 Testy integracyjne (Integration Testing)

Testy integracyjne weryfikują działanie systemu jako całości. Wykorzystano tu klienta testowego Flaska (`test_client`), który symuluje wysyłanie żądań HTTP do aplikacji.

Aby testy nie wpływały na produkcyjną bazę danych, zastosowano mechanizm *fixture* (w pliku `conftest.py`), który na czas testu tworzy pustą bazę w pamięci RAM (`sqlite:///memory:`).

```

from app.routes.labor_code_validator import LaborCodeValidator
from datetime import datetime, timedelta

def test_daily_rest_violation():
    """
    Test sprawdza, czy algorytm wykrywa brak 11h przerwy między zmianami.
    """

    shift_1_end = datetime(2023, 10, 1, 22, 0, 0)
    shift_2_start = datetime(2023, 10, 2, 6, 0, 0)

    is_valid, error_msg = LaborCodeValidator.check_daily_rest(shift_1_end, shift_2_start)

    assert is_valid is False
    assert "Naruszenie odpoczynku dobowego" in error_msg

```

Rysunek 5.2: Test walidatora odpoczynku dobowego, Źródło: Opracowanie własne.

Konfiguracja środowiska testowego:

```

import pytest
from app import create_app, db

@pytest.fixture
def client():

    app = create_app(config_name='testing')
    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///memory:'

    with app.test_client() as client:
        with app.app_context():
            db.create_all()
            yield client
            db.drop_all()

```

Rysunek 5.3: Konfiguracja fixture w pytest, Źródło: Opracowanie własne.

Rysunek 5.4: Test endpointu logowania (API)

Poniższy test symuluje pełną ścieżkę: rejestrację użytkownika w bazie, wysłanie żądania POST do /auth/login i sprawdzenie, czy serwer zwraca poprawny token JWT.

```
from app.models import User, db

def test_login_successful(client):
    """
    Test sprawdza endpoint /auth/login dla poprawnego użytkownika.
    """
    user = User(email="admin@hr.pl", name="Admin", role="admin")
    user.set_password("admin123")
    db.session.add(user)
    db.session.commit()
    response = client.post('/auth/login', json={
        'email': 'admin@hr.pl',
        'password': 'admin123'
    })
    assert response.status_code == 200
    data = response.get_json()
    assert 'access_token' in data
    assert data['role'] == 'admin'
```

Rysunek 5.4: Test integracyjny procesu logowania, Źródło: Opracowanie własne.

Rysunek 5.5: Test zabezpieczeń (RBAC)

Ważnym aspektem testów integracyjnych jest sprawdzenie, czy system blokuje dostęp nieuprawnionym użytkownikom.

```
def test_access_denied_without_token(client):
    """
    Test sprawdza, czy próba pobrania listy pracowników bez logowania
    kończy się błędem 401.
    """
    response = client.get('/users/employees')

    assert response.status_code == 401
```

Rysunek 5.5: Test odmowy dostępu do zasobów chronionych. Źródło: Opracowanie własne

5.4 Testy eksploracyjne

Testy eksploracyjne to metoda weryfikacji oprogramowania, która polega na manualnym badaniu aplikacji bez wcześniej przygotowanego, sztywnego scenariusza testowego. Podejście to opiera się na intuicji oraz doświadczeniu testera, co pozwala na wykrycie nietypowych błędów i przypadków brzegowych, które mogły zostać pominięte w testach jednostkowych i automatycznych.

W projekcie „HRmonogram” testy eksploracyjne były wykonywane iteracyjnie, po wdrożeniu każdej nowej funkcjonalności. Pozwoliły one na zidentyfikowanie istotnych błędów logicznych oraz problemów z integralnością danych, takich jak:

- **Konflikty w harmonogramach pracowniczych:**

Wykryto możliwość przypisania tego samego pracownika do dwóch różnych grafików trwających w tym samym przedziale czasowym, co prowadziło do przekłamania w ewidencji czasu pracy.

- **Błąd unikalności nazw departamentów:**

System wymuszał globalną unikalność nazw działów w całej bazie danych, co uniemożliwiało utworzenie departamentu o nazwie (np. „Kuchnia” lub „Recepcja”) w dwóch niezależnych obiektach.

- **Utrata spójności danych historycznych (Problem konfiguracji):**

Przed wprowadzeniem mechanizmu *Snapshot*, zmiana globalnych preferencji obiektu (np. edycja godzin otwarcia lub skrótów zmian) powodowała błędne wyświetlanie i przeliczanie grafików, które zostały już wcześniej zaakceptowane i zarchiwizowane.

- **Desynchronizacja siatki kalendarza:**

Wykryto brak blokady edycji dat w utworzonym już grafiku. Zmiana miesiąca lub roku w nagłówku istniejącego harmonogramu nie aktualizowała przypisanych zmian, co powodowało, że dni tygodnia nie pokrywały się z dniami miesiąca (np. zmiana przypisana do poniedziałku wyświetlała się pod datą wypadającą we wtorek).

Wszystkie powyższe nieprawidłowości zostały skorygowane przed finalną wersją systemu, m.in. poprzez wdrożenie walidacji międzyobiektowej oraz wzorca *Snapshot* dla ustawień grafiku.

Rozdział 6

Interfejs użytkownika

Rozdział 7

Podsumowanie i wnioski

Dodatki

Źródła

Lista dodatkowych plików, uzupełniających tekst pracy

Spis rysunków

1.1	Grafik Optymalny (Źródło: https://www.grafikoptymalny.pl/ , Data dostępu: 5.12.2025)	6
1.2	When I Work (Źródło: https://wheniwork.com/ ,Data dostępu: 5.12.2025)	8
2.1	Diagram przypadków użycia, Źródło: Opracowanie własne	23
4.1	Struktura projektu backendu. Źródło: Opracowanie własne	34
4.2	Struktura projektu frontend (React), Źródło: Opracowanie własne.	37
4.3	Definicja modelu User (plik backend/app/models.py), Źródło: Opracowanie własne	39
4.4	Definicja modelu pojedynczej zmiany (plik backend/app/models.py), Źródło: Opracowanie własne	39
4.5	Diagram ERD projektu bazy danych, Źródło: Opracowanie własne	41
4.6	Moduły routingu w aplikacji Flask. Źródło: Opracowanie własne	41
5.1	Test jednostkowy modelu User, Źródło: Opracowanie własne.	46
5.2	Test walidatora odpoczynku dobowego, Źródło: Opracowanie własne.	47
5.3	Konfiguracja fixture w pytest, Źródło: Opracowanie własne.	47
5.4	Test integracyjny procesu logowania, Źródło: Opracowanie własne.	48
5.5	Test odmowy dostępu do zasobów chronionych. Źródło: Opracowanie własne	48

Spis tabel

4.1	Wykaz kluczowych końcówek API systemu HRmonogram, Źródło: Opracowanie własne.	42
-----	---	----